DỰ ÁN TỰ ĐỘNG HÓA BOSOL AI
BỘ PHẬN KỸ THUẬT & CÔNG NGHỆ THÔNG TIN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

ĐẶC TẢ DỰ ÁN: HỆ THỐNG AI ĐỌC BẢN VẼ KỸ THUẬT & TÀI LIỆU HỖN HỢP

TÀI LIỆU KỸ THUẬT TRIỂN KHAI PHIÊN BẢN v2.8 (CAD ENGINE CỐT LÕI & ĐỊNH HƯỚNG COPILOT/RAG v2.9) - CẬP NHẬT 24/10/2026

THÔNG TIN VĂN BẢN	QUY ĐỊNH CHI TIẾT & THAM SỐ ÁP DỤNG
Tên tài liệu	Đặc tả kỹ thuật Hệ thống AI đọc bản vẽ kỹ thuật cơ khí & tài liệu hỗn hợp đa định dạng
Phiên bản chuẩn	**v2.8 / v2.9 Master Blueprint** (Khớp Lộ trình 24/10/2026: CAD Engine là phạm vi triển khai hiện tại, Chatbot Copilot là bước tiếp theo, RAG nâng cao là định hướng mở rộng)
Tài liệu quy chiếu tiến độ	Lo_trinh_phat_trien_va_tien_do_du_an.pdf / Lo_trinh_phat_trien_va_tien_do_du_an.md (Bảng theo dõi tiến độ chi tiết v2.8)
Tiến độ tổng thể	**Đã hoàn thành ~65% - 70%** khối lượng công việc nền tảng cốt lõi (Toán học tọa độ Affine, Text-first PyMuPDF, PostgreSQL DB Queue, Web UI Split-View). Đang bước vào giai đoạn nâng cấp v2.8 (Chuẩn hóa Schema, Multi-OCR, Gemini Rescue Layer, AI Router).
Quy ước đánh dấu tiến trình	`[x] ĐÃ HOÀN THÀNH` : Đã code, đã chạy kiểm thử, có kết quả đầu ra thực tế. `[/] ĐANG CẢI TIẾN / REFACTOR` : Đã có mã nguồn nhưng cần sửa đổi/đồng bộ chuẩn v2.8. `[ ] CHƯA LÀM (KẾ HOẠCH)` : Hạng mục mới cần lập trình bổ sung tiếp theo.
Mô hình vận hành	**Local-First Hybrid Architecture (Xác định rõ phân kỳ triển khai):** • *Phạm vi triển khai hiện tại (CAD Engine):* Bóc tách Khung tên, BOM từng ô với BBox, Kích thước Ø/±; Web Review Split-View 50/50; xuất dữ liệu ERP khi kỹ sư duyệt. • *Bước hoàn thiện tiếp theo (Chatbot Copilot):* Trợ lý ảo sử dụng Ollama Text chuẩn hóa JSON và trả lời câu hỏi dựa trên dữ liệu trung gian chuẩn đã trích xuất từ bản vẽ. • *Định hướng mở rộng tương lai (Document RAG nâng cao - Chưa thuộc phạm vi nghiệm thu v2.8):* Đọc tài liệu hỗn hợp (PDF, Scan, Excel, Word, PPT) $\rightarrow$ Hybrid Search (BM25 + BGE-M3) $\rightarrow$ Reranker (BGE-Reranker-v2-m3) $\rightarrow$ Chatbot trích nguồn kiểm chứng.
Mô hình AI cốt lõi	**Local AI cho bóc tách & chuẩn hóa:** Dùng **Ollama Text** như lộ trình để chuẩn hóa JSON và chạy chatbot Copilot dựa trên dữ liệu trích xuất; cấu hình **Qwen2.5-7B** chỉ là cấu hình thử nghiệm, chưa xác nhận khả năng chạy ổn định lâu dài trên GPU 6GB (cần đo kiểm VRAM & benchmark). Multi-OCR gồm PaddleOCR CPU làm chính, Tesseract (vie+eng) dự phòng. **Cloud Rescue (Tầng cứu viện API):** Giữ `GEMINI_MODEL_ID` **cấu hình động** (mặc định chọn model Flash thế hệ mới đang được Google hỗ trợ chính thức tại thời điểm triển khai, ví dụ `gemini-2.5-flash`; không dùng cố định các model cũ như `gemini-1.5-flash` hay bản thử nghiệm `gemini-2.0-flash` nếu đã hết hạn hỗ trợ) qua SDK chính thức `google-genai`. Áp dụng nghiêm ngặt chính sách User Consent và kiểm soát ngân sách theo lộ trình; model chính thức và mức chi phí chỉ là giả định tính toán cần đo kiểm xác minh khi triển khai thực tế trên tập crop mờ.
Hạ tầng & Cơ sở dữ liệu	**0 VNĐ thuê máy chủ** (Chạy All-in-One trên 01 Laptop nội bộ có GPU 6GB). **PostgreSQL 16** là kho dữ liệu bắt buộc hiện tại (quản lý Job Queue `FOR UPDATE SKIP LOCKED`, mô hình trạng thái kép Dual-State, Duyệt BOM từng ô, ERP sync). **ChromaDB Local** chỉ đưa vào khi thực hiện phân hệ RAG nâng cao trong tương lai.
Nguyên tắc sống còn	**Tuyệt đối không bịa (Zero Blind Trust & Zero Hallucination):** Tách bạch Dual-State (máy xử lý `processing_status` vs kỹ sư duyệt `review_status`); khóa cứng nút xuất ERP cho đến khi kỹ sư thực hiện hành động phê duyệt chủ động và backend kiểm tra hợp lệ; Ngưỡng từ chối điểm liên quan score < 0.3 (ngưỡng thử nghiệm giả định ban đầu cho phân hệ RAG mở rộng, cần hiệu chuẩn trên Golden QA; không áp dụng cho CAD Copilot hiện tại); Chatbot đối chiếu trực tiếp trên dữ liệu trung gian chuẩn hoặc trích nguồn kiểm chứng.

MỤC LỤC TỔNG QUAN TÀI LIỆU (15 PHẦN)

1. **PHẦN 1.** Phạm vi dự án và kiến trúc tổng thể (Trọng tâm CAD Engine hiện tại, Chatbot Copilot & Định hướng RAG mở rộng)
2. **PHẦN 2.** Hạ tầng All-in-One trên 01 Laptop và cơ chế hàng đợi Worker (Direct DB Queue)
3. **PHẦN 3.** Tầng AI: Ollama Text Local, Gemini Rescue Layer & SDK `google-genai` chuẩn
4. **PHẦN 4.** Cơ sở dữ liệu Dual-State: Quản lý bóc tách CAD, PostgreSQL 16 & Định hướng ChromaDB mở rộng
5. **PHẦN 5.** Toán học hệ tọa độ: Chuẩn hóa PyMuPDF BBox, Ma trận biến đổi PDF.js và giới hạn kiểm chứng Fixtures (7/7 tests mẫu theo báo cáo)
6. **PHẦN 6.** Tầng đọc file (Ingestion): Router 4 nhánh PDF bản vẽ, Multi-OCR và Bộ đọc Office mở rộng
7. **PHẦN 7.** Tầng Retrieval & RAG: Hybrid Search (BM25 + BGE-M3), Reranker (Định hướng mở rộng tương lai)
8. **PHẦN 8.** Cơ chế sinh câu trả lời: Ollama Text Copilot trên dữ liệu trung gian & Prompt chống bịa
9. **PHẦN 9.** Quy trình hệ thống toàn diện từ đầu đến cuối & Nguyên tắc Kỹ sư phê duyệt trước khi xuất ERP
10. **PHẦN 10.** Cấu trúc thư mục mã nguồn nâng cấp và đặc tả các module cốt lõi
11. **PHẦN 11.** Cơ chế cô lập sự cố Subprocess, Kế hoạch Circuit Breaker và Quản lý Timeout/Retry cho Gemini API
12. **PHẦN 12.** Schema dữ liệu: BOM cấp từng ô, Candidates đối chiếu bình đẳng và RAG Metadata
13. **PHẦN 13.** Phân tích chi phí định lượng & Bài toán giả định ngân sách (Gemini API Rescue Layer)
14. **PHẦN 14.** Kế hoạch kiểm thử (Benchmark CAD 4 Cấu hình trên 40 PDF + Chỉ số RAG) & Lộ trình thực hiện chi tiết 4 Giai đoạn
15. **PHẦN 15.** Kết luận và các quyết định thiết kế bất biến (Local-First Hybrid Architecture)

BẢNG TỔNG HỢP CÁC NÂNG CẤP ĐỘT PHÁ Ở PHIÊN BẢN v2.8/v2.9 UNIFIED

Thành phần nâng cấp	Nội dung cải tiến & Phân kỳ triển khai (Cập nhật 24/10/2026)	Giá trị kỹ thuật & Nghiệp vụ thực tế
1. Phân định rõ phạm vi	Xác định **CAD Engine** là phạm vi triển khai và nghiệm thu hiện tại; **Chatbot Copilot** (dựa trên dữ liệu trung gian) là bước tiếp theo; **RAG nâng cao** là định hướng mở rộng (chưa thuộc phạm vi nghiệm thu v2.8).	Tập trung nguồn lực giải quyết dứt điểm bài toán cốt lõi bóc tách bản vẽ kỹ thuật phục vụ sản xuất và nhập kho ERP.
2. Nâng cấp LLM Local	Sử dụng **Ollama Text** theo lộ trình để chuẩn hóa JSON và trả lời chat; cấu hình **Qwen2.5-7B** chỉ là cấu hình thử nghiệm, cần kiểm chứng VRAM và độ ổn định trên GPU 6GB.	Đảm bảo an toàn tài nguyên máy trạm, không gây tràn VRAM, tối ưu hóa tốc độ chuẩn hóa dữ liệu bản vẽ.
3. Cứu viện Gemini API	Giữ `GEMINI_MODEL_ID` **cấu hình động** (mặc định `gemini-2.5-flash` hoặc model Flash đang active theo tài liệu Google GenAI SDK tại thời điểm triển khai; không fix cứng model cũ) qua SDK `google-genai`; chính sách consent và ngân sách chặt chẽ; chi phí chỉ là giả định tính toán cần xác minh khi triển khai.	Kiểm soát tối đa tính bảo mật và ngân sách doanh nghiệp; cứu viện chính xác các ô mờ, số đo khó mà OCR cục bộ đọc sai.
4. Cơ sở dữ liệu cốt lõi	**PostgreSQL 16** là kho dữ liệu hiện tại duy nhất bắt buộc (quản lý Job Queue, mô hình Dual-State, BOM từng ô, ERP sync); **ChromaDB Local** chỉ đưa vào khi thực hiện RAG nâng cao.	Đơn giản hóa kiến trúc hạ tầng cục bộ MVP, bảo toàn tính nguyên tử dữ liệu và hỗ trợ tốt kiểm duyệt đa người dùng.
5. Ingestion & Multi-OCR	Router 4 nhánh dựa trên text density, vector primitives và raster image; Multi-OCR (PaddleOCR CPU chính + Tesseract dự phòng); đọc Office (Excel, Word, PPT) là định hướng mở rộng RAG.	Mục tiêu đạt độ chính xác phân loại cao trên các loại tài liệu bản vẽ kỹ thuật (theo báo cáo nội bộ đạt 15/15 file trên tập mẫu thử nghiệm; cần tiếp tục xác nhận độc lập trên tập bản vẽ thực tế mở rộng); giảm thiểu tối đa sai sót nhầm lẫn ký tự số đo cơ khí.
6. Cơ chế chống ảo giác & An toàn ERP	Tách bạch Dual-State (máy xử lý vs kỹ sư duyệt); BOM từng ô có BBox và ảnh crop; **khóa nút xuất ERP** cho đến khi kỹ sư thực hiện hành động phê duyệt chủ động và backend kiểm tra hợp lệ.	Kiểm soát và hạn chế tối đa rủi ro đưa số đo sai hoặc bịa đặt vào dây chuyền sản xuất; kỹ sư luôn là người phê duyệt cuối cùng trước khi cho phép xuất dữ liệu.
7. Kế hoạch kiểm thử	Trọng tâm là **Benchmark 4 cấu hình trên 40 PDF bản vẽ thực tế** (accuracy, P50/P95 latency, chi phí USD, VRAM); bộ chỉ số đo lường RAG (Hit Rate, Faithfulness) áp dụng khi kích hoạt phân hệ RAG.	Đo kiểm định lượng khoa học hiệu năng và tính ổn định của hệ thống trước khi nghiệm thu bàn giao xưởng.

PHẦN 1. PHẠM VI DỰ ÁN VÀ KIẾN TRÚC TỔNG THỂ (TRỌNG TÂM CAD ENGINE HIỆN TẠI, CHATBOT COPILOT & ĐỊNH HƯỚNG RAG MỞ RỘNG)

Hệ thống được thiết kế theo triết lý **Local-First Hybrid Architecture**, phân định rõ ràng ranh giới giữa **phạm vi triển khai hiện tại** theo Lộ trình v2.8 và **định hướng mở rộng tương lai**:

1. **Phân hệ CAD Engine (Phạm vi triển khai cốt lõi hiện tại - Giai đoạn 1 đến 3):** Tập trung giải quyết bài toán trọng tâm của nhà máy sản xuất: bóc tách tự động dữ liệu hình học, bảng kê vật tư (BOM từng ô kèm Bounding Box và Evidence Crop), và khung tên từ 4 loại PDF bản vẽ cơ khí; cung cấp giao diện kiểm duyệt Web Review Split-View 50/50 hai chiều và khóa xuất dữ liệu ERP cho đến khi kỹ sư duyệt hoàn tất.
2. **Chatbot Copilot Bản vẽ (Bước hoàn thiện tiếp theo - Giai đoạn 4):** Sử dụng **Ollama Text** để chuẩn hóa cấu trúc JSON và đóng vai trò trợ lý thông minh trả lời các câu hỏi kỹ thuật của kỹ sư dựa trên **dữ liệu trung gian chuẩn đã được trích xuất** từ bản vẽ (Title Block, BOM, Kích thước).
3. **Phân hệ Document RAG nâng cao (Định hướng mở rộng tương lai - Chưa thuộc phạm vi nghiệm thu v2.8):** Kiến trúc dự kiến nhằm mở rộng năng lực đọc hiểu toàn diện kho tài liệu kỹ thuật hỗn hợp (PDF scan, bảng tính Excel bảo toàn dòng/sheet, tài liệu Word/PowerPoint), kết hợp Hybrid Search (BM25 + BGE-M3), Reranker và ChromaDB. Phân hệ này được thiết kế sẵn sàng về mặt kiến trúc nhưng chưa nằm trong phạm vi nghiệm thu của giai đoạn hiện hành.

1.1. Ma trận phân định tính năng và phân kỳ triển khai

Phân hệ & Tính năng	Phân kỳ & Trạng thái	Dữ liệu / Tài liệu áp dụng	Đầu ra nghiệp vụ & Mục tiêu
Khung tên bản vẽ (Title Block)	[x] ĐÃ XONG (Text) [ ] CHƯA LÀM (Scan)	• Mã bản vẽ (`drawing_number`) • Tên chi tiết (`part_name`) • Vật liệu (`material`), Tỷ lệ	Tự động định danh hồ sơ, lập mã sản phẩm trong kho vật tư và quản lý phiên bản thiết kế.
Bảng kê BOM cấp từng ô	[/] ĐANG CẢI TIẾN (Mục 2.2)	• Số thứ tự (`item`), Mã (`part_number`) • Số lượng (`quantity`), Vật liệu (`material`) • **BBox, Evidence crop từng ô**	Bóc tách chính xác từng ô dữ liệu; hiển thị trên Web Review để kỹ sư đối chiếu và khóa xuất ERP.
Kích thước & Dung sai (Dimensions)	[x] ĐÃ XONG (Text) [ ] CHƯA LÀM (Rescue)	• Kích thước bao (Dài x Rộng x Cao) • Đường kính trục/lỗ (`Ø`), Bán kính (`R`) • Dung sai (`±` , lệch trên/dưới)	Xác định kích thước phôi để tính giá gia công nhanh và chỉ định máy CNC phù hợp.
Chatbot Copilot Bản vẽ	[/] ĐANG CẢI TIẾN (Mục 4.1)	• Trợ lý hỏi đáp trên **dữ liệu trung gian đã trích xuất** (Title Block, BOM, Kích thước) • Chuẩn hóa JSON bằng Ollama Text	Kỹ sư tra cứu nhanh thông số bản vẽ ngay trên màn hình Split-View; không nạp vision nặng gây tràn GPU.
Tra cứu RAG tài liệu kỹ thuật & Hợp đồng	[MỞ RỘNG TƯƠNG LAI] (Chưa nghiệm thu v2.8)	• Thuyết minh thiết kế, tiêu chuẩn nghiệm thu • Hợp đồng, biên bản kiểm tra chất lượng	Hybrid Search (BM25 + BGE-M3) + Reranker tra cứu đa tài liệu kèm trích nguồn số trang.
Bóc tách & Tra cứu Bảng tính Excel	[MỞ RỘNG TƯƠNG LAI] (Chưa nghiệm thu v2.8)	• Bảng định mức tiêu hao nguyên vật liệu • Bảng kê đơn giá khoán dạng Excel	Truy vấn số liệu chính xác theo từng hàng, từng cột mà không bị biến dạng bảng; dẫn chiếu số dòng và sheet.

1.2. Sơ đồ kiến trúc toàn trình hệ thống (Phân định trạng thái thực tế & Ưu tiên CAD Pipeline)

```
flowchart TD
    INPUT["📥 TÀI LIỆU ĐẦU VÀO
    (Bản vẽ CAD Vector, Bản vẽ Scan mờ, PDF Văn bản)"] --> DOC_ROUTER

    subgraph INGESTION["TẦNG ĐỌC VÀ TIỀN XỬ LÝ (INGESTION LAYER - LOCAL 100%)"]
        DOC_ROUTER["🧭 document_router.py [x] ĐÃ XONG (100% trên 15 file test)
        (Phân loại 4 nhánh: text density, vector primitives, raster image)"]
        DOC_ROUTER -->|"Nhánh 1: PDF CAD Vector / Text"| P1["⚡ PyMuPDF text_extractor.py [x] ĐÃ XONG
        (15ms - 76ms, 0 VRAM, bóc Text Vector & BBox)"]
        DOC_ROUTER -->|"Nhánh 2 & 3: PDF Scan / Nét mờ"| P2["📑 ocr_adapter.py [ ] CHƯA LÀM (Mục 2.4)
        (PaddleOCR CPU chính + Tesseract vie/eng dự phòng)"]
    end

    P1 --> IR_CACHE["💾 Lưu Trữ Trung Gian & Cache Phiên Bản
    evidence_store.py [ ] CHƯA LÀM (Mục 2.5)
    sha256(file_hash + page + pipeline_ver + config_hash)"]
    P2 --> IR_CACHE

    subgraph CORE_CAD["⭐⭐ TRỌNG TÂM TRIỂN KHAI: PIPELINE CAD ENGINE & KIỂM DUYỆT ERP ⭐⭐"]
        IR_CACHE --> DRAWING_PARSER["📐 drawing_parser.py [/] ĐANG CẢI TIẾN
        (Bóc tách Title Block, BOM từng ô, Kích thước Ø/±)"]
        DRAWING_PARSER --> AI_ROUTER_CAD{"ai_router.py [ ] CHƯA LÀM (Mục 3.3)
        OCR conf < 0.8 & User Consent & Budget < $2?"}

        AI_ROUTER_CAD -->|"Thỏa mãn 3 điều kiện"| GEMINI_RESCUE["☁️ gemini_adapter.py [ ] CHƯA LÀM (Mục 3.1 & 3.2)
        Google Gemini API (GEMINI_MODEL_ID động)
        SDK google-genai + Circuit Breaker (Timeout 15s)"]
        AI_ROUTER_CAD -->|"Không thỏa mãn / Đã rõ nét"| CANDIDATES_STORE

        GEMINI_RESCUE -->|"Lưu ứng viên đối chiếu bình đẳng"| CANDIDATES_STORE

        CANDIDATES_STORE["📋 Schema BOM từng ô [/] ĐANG CẢI TIẾN (Mục 2.2)
        suggested_val, verified_val, bbox_pdf, source, evidence_crop_id"]

        CANDIDATES_STORE --> POSTGRES_DB["🗄️ PostgreSQL 16 [x] ĐÃ XONG (3/3 Tests OK)
        (Job Queue FOR UPDATE SKIP LOCKED, Lease 300s, Reaper)"]
        POSTGRES_DB --> DUAL_STATE["🚦 Mô hình Trạng thái Kép [/] ĐANG CẢI TIẾN (Mục 2.1)
        processing_status (máy) & review_status (kỹ sư)"]

        DUAL_STATE --> WEB_SPLITVIEW["🔒 Web Review Split-View 50/50 [x] ĐÃ XONG
        (PDF.js Highlight bbox, Discrepancy Card chọn nhanh [A]/[B])"]

        WEB_SPLITVIEW -->|"CHỈ MỞ KHÓA KHI review_status == 'approved'"| ERP_EXPORT["🚀 Xuất Dữ Liệu Sản Xuất / ERP [ ]
        (Mục 4.2)"]
    end

    subgraph COPILOT_STEP["BƯỚC TIẾP THEO: OLLAMA TEXT CHUẨN HÓA & CHATBOT COPILOT (GIAI ĐOẠN 4)"]
        POSTGRES_DB -.->|"Dữ liệu trung gian chuẩn"| OLLAMA_TEXT["🦙 Ollama Text (Qwen2.5) [/] ĐANG CẢI TIẾN (Mục 4.1)
        (Chuẩn hóa JSON & Trợ lý hỏi đáp trên dữ liệu bản vẽ đã bóc tách)"]
        OLLAMA_TEXT --> CHAT_COPILOT_UI["💬 Chatbot Copilot Web UI (Nửa trái màn hình Split-View)"]
    end

    subgraph EXTENSION_RAG["ĐỊNH HƯỚNG MỞ RỘNG TƯƠNG LAI: DOCUMENT RAG NÂNG CAO (CHƯA NGHIỆM THU v2.8)"]
        OFFICE_IN["📄 office_extractor.py (Excel, Word, PPT)"] -.-> RAG_CHUNKER["✂️ Semantic Chunker (Giữ bảng & Header row)"]
        RAG_CHUNKER -.-> BGE_M3["🧠 Embedding BAAI/bge-m3 + BM25 Keyword"]
        BGE_M3 -.-> CHROMA_STORE["💾 ChromaDB Local Persistent (Vector Store)"]
        CHROMA_STORE -.-> RERANK["🎯 bge-reranker-v2-m3
        (Ngưỡng thử nghiệm >= 0.3 - cần hiệu chuẩn trên Golden QA)"]
        RERANK -.-> RAG_GEN["💬 Trả lời câu hỏi tài liệu hỗn hợp kèm Trích nguồn [file, trang/sheet/dòng]"]
    end
```

PHẦN 2. HẠ TẦNG ALL-IN-ONE TRÊN 01 LAPTOP VÀ CƠ CHẾ HÀNG ĐỢI WORKER (DIRECT DB QUEUE)

Để đảm bảo **chi phí thuê máy chủ là 0 VNĐ** và dữ liệu bản vẽ bảo mật tuyệt đối bên trong nhà xưởng, toàn bộ hệ thống được tối ưu hóa để vận hành an toàn trên **01 Laptop kỹ thuật thông dụng** cấu hình tiêu chuẩn: CPU Intel Core i7 / AMD Ryzen 7, RAM 16GB, Card đồ họa GPU NVIDIA 6GB VRAM (RTX 3060/4050 Laptop trở lên).

2.1. Chiến lược phân bổ tài nguyên MVP (Tránh nghẽn RAM 16GB & GPU 6GB)

Trong giai đoạn MVP, để đảm bảo hệ thống chạy mượt mà và không xảy ra tình trạng tràn bộ nhớ (Out-Of-Memory - OOM) hay treo máy trạm, hệ thống áp dụng nguyên tắc **chạy ít worker và không nạp đồng thời nhiều mô hình nặng**:

- **OCR Engine chạy ít Worker:** Chỉ sử dụng **1–2 worker CPU** để xử lý tuần tự từng trang bản vẽ; không bật 4–6 worker ồ ạt làm cạn kiệt RAM và băng thông CPU của hệ điều hành.
- **Điều phối mô hình tuần tự:** Khi pipeline CAD đang chạy bóc tách hình ảnh, tuyệt đối không nạp đồng thời các mô hình Embedding (bge-m3) hay Reranker.
- **Ollama Text nạp theo nhu cầu:** Mô hình Ollama Text chỉ được gọi tuần tự khi cần chuẩn hóa JSON sau OCR hoặc khi kỹ sư mở cửa sổ Chatbot Copilot; không duy trì thường trực trên VRAM nếu máy cần ưu tiên xử lý đồ họa bản vẽ nặng.

Thành phần tiến trình	Công nghệ & Kích thước model	Mức tiêu thụ VRAM / RAM	Phương thức tối ưu hóa MVP
LLM Local (Ollama Text)	Ollama Text (Qwen2.5-7B là cấu hình thử nghiệm)	~4.0 - 4.2 GB VRAM (khi nạp)	Quantization 4-bit; nạp theo nhu cầu; cần đo kiểm VRAM thực tế trước khi chốt cố định.
OCR Engines	PaddleOCR CPU (chính) + Tesseract 5 (dự phòng)	0 MB VRAM (~1.0 - 1.5 GB RAM)	Chỉ chạy 1–2 CPU worker tuần tự, giải phóng 100% GPU cho các tác vụ hiển thị và AI.
Cơ sở dữ liệu & Backend	PostgreSQL 16 + FastAPI	~1.0 - 1.2 GB RAM hệ thống	Thành phần cốt lõi hiện tại; chạy ngầm nhẹ nhàng dưới dạng service cục bộ.
RAG Models (Mở rộng)	ChromaDB + BAAI/bge-m3 + Reranker	~1.2 GB VRAM / ~1 GB RAM	Chỉ kích hoạt khi triển khai phân hệ RAG mở rộng; không nạp cùng lúc với CAD pipeline.

2.2. Cơ chế nhận việc Direct DB Queue Worker (PostgreSQL FOR UPDATE SKIP LOCKED)

Để tối ưu cho mô hình All-in-One trên 1 máy đơn, hệ thống cung cấp giải pháp **Direct DB Queue**: Worker daemon kết nối trực tiếp vào PostgreSQL nội bộ qua connection pool để lấy job, gia hạn lease và xử lý, loại bỏ độ trễ và sự phụ thuộc vào tầng mạng HTTP nội bộ.

Kết quả đo kiểm nội bộ theo báo cáo: Bộ kiểm thử `tests/test_database_and_worker.py` được tài liệu báo cáo đạt **3/3 test cases kịch bản mẫu** (nhận việc nguyên tử `SKIP LOCKED`, gia hạn Lease 300s, và chống ghi đè xung đột). *Lưu ý:* Đây là kết quả ghi nhận từ báo cáo kiểm thử nội bộ trên tập ca kiểm thử mẫu; hiện chưa có mã nguồn đầy đủ và log kiểm thử độc lập đính kèm để thẩm định bên ngoài.

1. **Nhận việc nguyên tử (Atomicity):** Sử dụng mệnh đề `FOR UPDATE SKIP LOCKED` giúp worker nhận đúng 1 job duy nhất ngay cả khi có nhiều luồng tranh chấp.
2. **Cơ chế Lease 300 giây & Heartbeat:** Khóa job trong 300 giây; định kỳ worker gửi heartbeat gia hạn lease; nếu worker bị crash quá 300s, job tự động mở lại cho worker khác.
3. **Chống ghi đè xung đột (Conflict 409):** Kiểm soát token và phiên bản cập nhật, ngăn chặn 2 tiến trình cùng cập nhật đè kết quả lên nhau.
4. **Tiến trình Reaper dọn Zombie job:** Tự động quét và phục hồi các job bị treo do tiến trình con đột ngột tắt nguồn.

> **PHÂN ĐỊNH TRẠNG THÁI MÃ NGUỒN:**
> • **Lõi Database & SQL Queue:** Đã hoàn thành theo báo cáo nội bộ (`[x] ĐÃ XONG`), định nghĩa tại `app/db/database.py` và `app/db/models.py` .
> • **Tích hợp Worker Daemon:** Tệp `app/workers/laptop_worker.py` đang được cải tiến (`[/] ĐANG CẢI TIẾN` theo Mục 2.3 của Lộ trình) để hỗ trợ linh hoạt cả hai chế độ: Direct DB Queue (chạy máy đơn) và HTTP Gateway (chạy phân tán nhiều máy qua LAN).

```python
from sqlalchemy import text
from sqlalchemy.orm import Session

def claim_next_job_direct(db: Session, worker_id: str, lease_token: str) -> dict | None:
    """Worker lấy job trực tiếp từ PostgreSQL nội bộ không qua HTTP (3/3 test cases theo báo cáo)."""
    sql = text("""
        UPDATE jobs
        SET processing_status = 'processing',
            attempts = attempts + 1,
            worker_id = :worker_id,
            lease_token = :lease_token,
            lease_expires_at = NOW() + INTERVAL '300 seconds',
            updated_at = NOW()
        WHERE id = (
            SELECT id FROM jobs
            WHERE (processing_status = 'queued' OR (processing_status = 'processing' AND lease_expires_at < NOW()))
              AND attempts < 3
            ORDER BY priority DESC, created_at ASC
            LIMIT 1
            FOR UPDATE SKIP LOCKED
        )
        RETURNING id, document_id, attempts, lease_token;
    """)
    result = db.execute(sql, {"worker_id": worker_id, "lease_token": lease_token}).fetchone()
    db.commit()
    return dict(result._mapping) if result else None
```

PHẦN 3. TẦNG AI: OLLAMA TEXT LOCAL, TẦNG CỨU VIỆN GEMINI & SDK GOOGLE-GENAI CHUẨN

3.1. Phân công vai trò giữa AI Cục bộ (Local) và Tầng Cứu viện Đám mây (Cloud)

Mô hình AI	Vị trí triển khai	Nhiệm vụ theo Lộ trình kỹ thuật	Chi phí & Đánh giá ổn định
Ollama Text *(Thử nghiệm: Qwen2.5-7B)*	Cục bộ trên máy trạm (Offline 100%)	• Chuẩn hóa dữ liệu OCR sang JSON có cấu trúc. • Chatbot Copilot hỏi đáp trên **dữ liệu trung gian đã trích xuất** (Title Block, BOM, Kích thước). • Sửa lỗi JSON cú pháp (`json_repair`).	**0 VNĐ** *Lưu ý:* Qwen2.5-7B chỉ là cấu hình thử nghiệm, chưa xác nhận chạy ổn định lâu dài trên GPU 6GB; cần đo kiểm VRAM và độ trễ tại Benchmark GĐ 4.
Google Gemini API *(Cấu hình qua `GEMINI_MODEL_ID`)*	Cloud API qua SDK mới `google-genai`	• **Tầng cứu viện bản vẽ (Rescue Crop):** Đọc các ô mờ, số đo đè nét vẽ mà OCR địa phương bó tay (< 0.8). • Hoạt động dưới sự kiểm soát của `ai_router.py` (Consent + Budget).	**Cấu hình động:** Mặc định `gemini-2.5-flash` (hoặc model Flash thế hệ mới đang được Google hỗ trợ chính thức tại thời điểm triển khai; không fix cứng các model cũ đã ngưng dịch vụ). *Lưu ý:* Mức chi phí dự kiến chỉ là giả định tính toán, cần xác minh qua kiểm thử thực tế.

3.2. Kế hoạch triển khai Adapter Gemini qua SDK google-genai (Giai đoạn 3 - Chưa làm)

Theo Lộ trình kỹ thuật, module `app/pipeline/gemini_adapter.py` là hạng mục **GIAI ĐOẠN 3 CHƯA LÀM (`[ ] CHƯA LÀM`)** (Mục 3.1 & 3.2). Kế hoạch triển khai chi tiết bao gồm:

1. **Sử dụng SDK mới chính thức:** Tích hợp thư viện mới `google-genai` của Google (loại bỏ hoàn toàn SDK cũ `google-generativeai`), hỗ trợ xuất JSON có cấu trúc (Structured Outputs) qua Pydantic schema.
2. **Biến môi trường linh hoạt:** Cấu hình `GEMINI_MODEL_ID` động (mặc định `gemini-2.5-flash` - model đang được API hỗ trợ ổn định; loại bỏ các tên model cũ như `gemini-1.5-flash` hoặc bản thử nghiệm `gemini-2.0-flash` nếu không còn duy trì trên API), cho phép chuyển đổi model linh hoạt qua biến môi trường mà không cần sửa code.
3. **Kiểm soát Timeout đúng tầng Transport & Circuit Breaker 3 trạng thái:** Đặt cấu hình timeout ở tầng kết nối HTTP khi khởi tạo `genai.Client(http_options=types.HttpOptions(timeout=15.0))` (lưu ý đối soát đơn vị float giây hoặc int mili-giây theo tài liệu phiên bản SDK `google-genai` đang cài đặt), tuyệt đối không đặt sai vị trí bên trong `types.GenerateContentConfig`; Thiết kế Circuit Breaker 3 trạng thái (CLOSED, OPEN, HALF-OPEN) đếm lỗi sau khi đã hết số lần retry và hỗ trợ request thăm dò để

phục hồi; Retry 2 lần có chọn lọc chỉ với các lỗi mạng/timeout hoặc lỗi quá tải tạm thời (429/503), loại trừ hoàn toàn các lỗi client 4xx, sai API key hay sai schema.

4. **Kiểm chứng thực tế 10–20 crop mờ:** Bắt buộc chạy kịch bản kiểm thử `tests/test_gemini_rescue_crops.py` trên 10–20 ảnh crop mờ của tập bản vẽ scan thực tế để đo lường độ chính xác và ghi nhận chi phí thực tế trước khi kích hoạt trên diện rộng.

> ⚠ **LƯU Ý ĐẶC TẢ KIẾN TRÚC & QUY TẮC MÃ NGUỒN (P1):** Đoạn mã dưới đây là **mã minh họa khái niệm kiến trúc (Architectural Reference)** nhằm chuẩn hóa luồng logic, không xem là mã nguồn đã được xác thực hoàn chỉnh để sao chép chạy ngay. Toàn bộ mã chạy thực tế bắt buộc phải được chuyển thành các file module `.py` độc lập (`app/pipeline/gemini_adapter.py` , `app/core/circuit_breaker.py`) và phải có bộ kiểm thử tự động (unit & integration tests) kiểm tra độc lập trước khi tích hợp vào worker.

```
import os
import time
import socket
import logging
from enum import Enum
from google import genai
from google.genai import types
from google.genai.errors import APIError
from pydantic import BaseModel, Field
from tenacity import retry, stop_after_attempt, wait_exponential

logger = logging.getLogger(__name__)

class CircuitState(Enum):
    CLOSED = "CLOSED"         # Bình thường, cho phép gọi API
    OPEN = "OPEN"             # Đang ngắt kết nối do lỗi liên tiếp
    HALF_OPEN = "HALF_OPEN"   # Hết thời gian ngắt, cho phép 1 request thăm dò

class GeminiCircuitBreaker:
    """
    Circuit Breaker 3 trạng thái cho Gemini Cloud API:
    - CLOSED: Hoạt động bình thường.
    - OPEN: Tạm dừng gọi Cloud khi có 3 lần lỗi liên tiếp (tính sau khi hết lượt retry).
    - HALF_OPEN: Hết thời gian chờ (recovery_time_sec), cho phép 1 request thăm dò phục hồi.
    """
    def __init__(self, failure_threshold: int = 3, recovery_time_sec: int = 600):
        self.failure_threshold = failure_threshold
        self.recovery_time_sec = recovery_time_sec
        self.consecutive_failures = 0
        self.circuit_open_until = 0.0
        self.state = CircuitState.CLOSED
        self._half_open_probe_in_flight = False

    def is_available(self) -> bool:
        now = time.time()
        if self.state == CircuitState.OPEN:
            if now >= self.circuit_open_until:
                # Đã hết thời gian ngắt, chuyển sang HALF-OPEN để thăm dò
                self.state = CircuitState.HALF_OPEN
                self._half_open_probe_in_flight = False
                logger.info("CIRCUIT BREAKER -> HALF-OPEN: Thử nghiệm 1 request thăm dò.")
            else:
                return False

        if self.state == CircuitState.HALF_OPEN:
            if self._half_open_probe_in_flight:
                return False
            self._half_open_probe_in_flight = True
            return True

        return True

    def record_success(self):
        """Ghi nhận thành công: reset lỗi và đưa Circuit về CLOSED."""
        self.consecutive_failures = 0
        self.circuit_open_until = 0.0
        self.state = CircuitState.CLOSED
        self._half_open_probe_in_flight = False

    def record_failure(self):
        """
        Ghi nhận thất bại:
        Chỉ đếm sau khi request đã cạn kiệt toàn bộ số lần retry hoặc gặp lỗi server nghiêm trọng.
        """
        self._half_open_probe_in_flight = False
        if self.state == CircuitState.HALF_OPEN:
            # Request thăm dò thất bại -> tiếp tục OPEN thêm một chu kỳ recovery
```

```
            self.state = CircuitState.OPEN
            self.circuit_open_until = time.time() + self.recovery_time_sec
            logger.warning(
                f"CIRCUIT BREAKER: Request thăm dò thất bại, tiếp tục OPEN trong {self.recovery_time_sec}s!"
            )
            return

        self.consecutive_failures += 1
        if self.consecutive_failures >= self.failure_threshold:
            self.state = CircuitState.OPEN
            self.circuit_open_until = time.time() + self.recovery_time_sec
            logger.warning(
                f"CIRCUIT BREAKER KÍCH HOẠT (OPEN): Tạm ngắt Gemini Cloud trong {self.recovery_time_sec}s "
                f"do {self.consecutive_failures} tác vụ thất bại liên tiếp sau retry!"
            )

circuit_breaker = GeminiCircuitBreaker(failure_threshold=3, recovery_time_sec=600)

class DrawingCellRescueOutput(BaseModel):
    """Schema Pydantic chuẩn hóa dữ liệu trả về từ Gemini Rescue Layer."""
    detected_value: str = Field(description="Giá trị text/số đo đọc được trong ảnh crop")
    prefix: str | None = Field(default=None, description="Ký hiệu tiền tố như Ø, R, M, 4-Ø")
    tolerance_upper: float | None = Field(default=None, description="Dung sai trên")
    tolerance_lower: float | None = Field(default=None, description="Dung sai dưới")
    raw_visual_note: str = Field(description="Mô tả trực quan nét vẽ hoặc chữ xung quanh")

# Các ngoại lệ mạng/timeout tạm thời được phép retry
TRANSIENT_EXCEPTIONS = (
    TimeoutError,
    ConnectionError,
    socket.timeout,
)

def _is_retryable_error(exc: BaseException) -> bool:
    """
    Lọc lỗi có chọn lọc:
    - Chỉ retry khi gặp lỗi kết nối mạng, timeout hoặc mã 429/503 từ API Google.
    - Tuyệt đối KHÔNG retry các lỗi không thể khắc phục: 400 Bad Request, 401/403 Sai API Key,
      404 Not Found, hoặc lỗi kiểm tra Schema Pydantic.
    """
    if isinstance(exc, TRANSIENT_EXCEPTIONS):
        return True
    if isinstance(exc, APIError):
        return exc.code in {429, 500, 502, 503, 504}
    return False

def get_gemini_client(timeout_sec: float = 15.0) -> genai.Client:
    """
    Khởi tạo Client chuẩn với HttpOptions cấu hình ở đúng tầng HTTP Transport.
    Lưu ý: Kiểm tra đơn vị timeout theo phiên bản google-genai cài đặt
    (chuẩn SDK: float seconds hoặc integer milliseconds).
    """
    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        raise ValueError("GEMINI_API_KEY chưa được thiết lập trong biến môi trường.")

    # HttpOptions đặt tại Client Initialization, KHÔNG đặt trong GenerateContentConfig
    http_opts = types.HttpOptions(timeout=float(timeout_sec))
    return genai.Client(api_key=api_key, http_options=http_opts)

# Retry tối đa 2 lần với giãn cách lũy thừa (2s - 6s), chỉ kích hoạt cho lỗi tạm thời hợp lệ
@retry(
    stop=stop_after_attempt(2),
    wait=wait_exponential(multiplier=1, min=2, max=6),
    retry=_is_retryable_error,
    reraise=True
)
def _call_gemini_api_internal(client: genai.Client, model_id: str, crop_bytes: bytes, prompt_hint: str):
```

```
    """Thực thi gọi API có cấu hình Structured Outputs (GenerateContentConfig chỉ chứa logic mô hình)."""
    config = types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=DrawingCellRescueOutput,
        temperature=0.1
    )
    return client.models.generate_content(
        model=model_id,
        contents=[
            prompt_hint,
            types.Part.from_bytes(data=crop_bytes, mime_type="image/png")
        ],
        config=config
    )

def rescue_crop_with_gemini(crop_bytes: bytes, prompt_hint: str) -> DrawingCellRescueOutput:
    """Tầng điều phối cứu viện Gemini: Kiểm tra Circuit Breaker, gọi API và cập nhật trạng thái."""
    if not circuit_breaker.is_available():
        raise RuntimeError("CIRCUIT_BREAKER_OPEN: Gemini API đang tạm ngắt do lỗi liên tiếp. Chuyển fallback OCR.")

    model_id = os.getenv("GEMINI_MODEL_ID", "gemini-2.5-flash")
    client = get_gemini_client(timeout_sec=15.0)

    try:
        response = _call_gemini_api_internal(client, model_id, crop_bytes, prompt_hint)
        circuit_breaker.record_success()
        return DrawingCellRescueOutput.model_validate_json(response.text)
    except Exception as exc:
        # Chỉ đếm thất bại sau khi đã thử lại hết số lần retry hoặc gặp lỗi server nghiêm trọng
        circuit_breaker.record_failure()
        logger.error(f"Lỗi gọi Gemini Rescue (sau retry / timeout / server): {exc}")
        raise
```

PHẦN 4. CƠ SỞ DỮ LIỆU DUAL-STATE: QUẢN LÝ BÓC TÁCH CAD, POSTGRESQL 16 & ĐỊNH HƯỚNG CHROMADB MỞ RỘNG

Hệ thống xác định **PostgreSQL 16** là kho cơ sở dữ liệu nền tảng hiện tại duy nhất bắt buộc (chạy cục bộ tại `localhost:5432/bosol_db`), chịu trách nhiệm quản lý toàn bộ cấu trúc bản vẽ, hàng đợi Job Queue nguyên tử, lịch sử gọi AI và trạng thái phê duyệt của kỹ sư. Kho dữ liệu vector **ChromaDB Local Persistent** chỉ được đưa vào cấu hình khi triển khai phân hệ Document RAG nâng cao trong tương lai.

4.1. Mô hình Trạng thái Kép Độc lập (Dual-State Model - Đang cải tiến theo Mục 2.1)

Theo Lộ trình kỹ thuật, hệ thống đang thực hiện hạng mục **cải tiến mã nguồn (`[/] ĐANG CẢI TIẾN` theo Mục 2.1)** trong tệp `app/db/models.py`: thay thế cột `status` đơn lẻ trước đây bằng **hai trục trạng thái hoàn toàn độc lập** nhằm tách bạch giữa tiến trình máy tính và quyết định của con người:

- `processing_status` **(Trạng thái xử lý của máy / Worker):**
  `queued` (Chờ nhận việc) $\rightarrow$ `processing` (Đang chạy OCR/AI) $\rightarrow$ `completed` (Hoàn tất bóc tách) | `failed` (Gặp sự cố lỗi).
- `review_status` **(Trạng thái kiểm duyệt của Kỹ sư xưởng):**
  `pending` (Chưa kiểm tra) $\rightarrow$ `in_review` (Đang đối chiếu số đo trên UI) $\rightarrow$ `approved` (Đã bấm duyệt toàn bộ) | `rejected` (Bác bỏ bản vẽ).
- **Hai bảng nghiệp vụ mới cần tạo theo Mục 2.1:**
  • `user_consents`: Lưu chính sách kỹ sư cho phép gửi ảnh crop lên Gemini Cloud Vision (mặc định `allow_cloud_vision = FALSE`).
  • `ai_requests`: Lưu vết token, độ trễ và chi phí USD từng lần gọi API để kiểm soát ngân sách theo thời gian thực.

> **QUY TẮC AN TOÀN ERP BẤT BIẾN:** Nút bấm *"Xuất dữ liệu sang ERP / File Excel"* trên giao diện Web Review bắt buộc phải bị **KHÓA CỨNG** cho đến khi `review_status == 'approved'` (toàn bộ các ô dữ liệu BOM và kích thước đã được kỹ sư bấm chấp thuận). Máy tính không bao giờ có quyền tự ý đẩy dữ liệu chưa duyệt vào hệ thống sản xuất.

4.2. Khung Schema PostgreSQL 16 (Đang đồng bộ trong `app/db/models.py`)

```sql
-- 1. Bảng quản lý Job và Trạng thái kép Dual-State (Mục 2.1)
CREATE TABLE jobs (
    id VARCHAR(64) PRIMARY KEY,
    document_id VARCHAR(64) NOT NULL,
    priority INT DEFAULT 0,
    processing_status VARCHAR(32) NOT NULL DEFAULT 'queued', -- queued, processing, completed, failed
    review_status VARCHAR(32) NOT NULL DEFAULT 'pending',    -- pending, in_review, approved, rejected
    attempts INT DEFAULT 0,
    worker_id VARCHAR(64),
    lease_token VARCHAR(64),
    lease_expires_at TIMESTAMP WITH TIME ZONE,
    result_json JSONB,
    error_code VARCHAR(64),
    error_message TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    completed_at TIMESTAMP WITH TIME ZONE
);

CREATE INDEX idx_jobs_queue ON jobs (priority DESC, created_at ASC)
WHERE processing_status = 'queued';

CREATE INDEX idx_jobs_result_jsonb ON jobs USING gin (result_json);

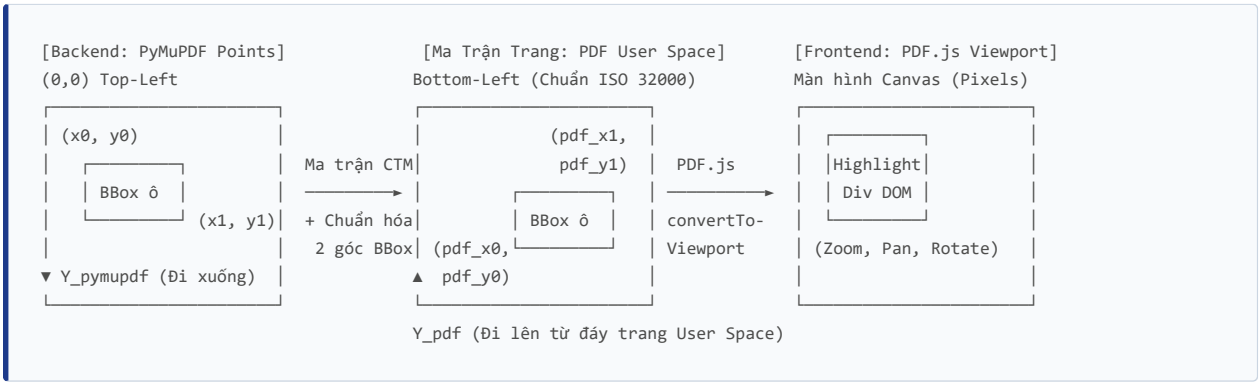
-- 2. Bảng quản lý sự đồng ý của kỹ sư khi gửi ảnh lên Cloud (Bảo mật theo Mục 2.1)
CREATE TABLE user_consents (
    id SERIAL PRIMARY KEY,
    document_id VARCHAR(64) NOT NULL,
    user_id VARCHAR(64) NOT NULL,
    allow_cloud_vision BOOLEAN DEFAULT FALSE,
    max_crops_allowed INT DEFAULT 10,
    granted_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. Bảng giám sát chi phí và lịch sử gọi API Gemini (Theo dõi ngân sách Mục 2.1)
CREATE TABLE ai_requests (
    id SERIAL PRIMARY KEY,
    job_id VARCHAR(64) NOT NULL,
    document_id VARCHAR(64) NOT NULL,
    crop_id VARCHAR(64) NOT NULL,
    ai_provider VARCHAR(32) NOT NULL,
    model_name VARCHAR(64) NOT NULL,
    prompt_tokens INT DEFAULT 0,
    completion_tokens INT DEFAULT 0,
    estimated_cost_usd NUMERIC(8, 6) DEFAULT 0.0,
    latency_ms INT NOT NULL,
    status VARCHAR(32) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

PHẦN 5. TOÁN HỌC HỆ TỌA ĐỘ: CHUẨN HÓA PYMUPDF BBOX, MA TRẬN BIẾN ĐỔI PDF.JS VÀ GIỚI HẠN KIỂM CHỨNG FIXTURES

Hệ thống xử lý tương thích giữa hai quy chuẩn tọa độ: **Hệ tọa độ PyMuPDF trong Backend (gốc tọa độ Top-Left [0,0], Y đi xuống theo khung nhìn trang)** và **Hệ tọa độ chuẩn PDF User Space mà PDF.js mong đợi (gốc tọa độ Bottom-Left [0,0], Y đi lên theo chuẩn ISO 32000-1)**.

Lưu ý kỹ thuật quan trọng về tính tổng quát (P0): Công thức đơn giản `pageHeight - y + cropY0` chỉ có thể đúng trong các trường hợp trang cơ bản (không xoay, MediaBox trùng CropBox). Tuy nhiên, trên bản vẽ kỹ thuật thực tế, các trang thường xuyên có thuộc tính xoay `/Rotate` (90°, 180°, 270°), có gốc tọa độ `MediaBox` khác [0,0], hoặc lề cắt `CropBox` lệch chuẩn. Việc lật trục thủ công sẽ dẫn đến sai lệch nghiêm trọng. Do đó, hệ thống bắt buộc phải sử dụng **phép biến đổi ma trận tọa độ thực tế của trang (Page Transformation Matrix / CTM)**, đồng thời **chuẩn hóa hai góc BBox** `[minX, minY, maxX, maxY]` trước khi đưa vào hàm chuyển đổi Viewport của PDF.js.



5.1. Thuật toán chuyển đổi tổng quát bằng Ma trận tọa độ trang trên PDF.js

Để đảm bảo tương thích tuyệt đối cho mọi bản vẽ có `CropBox`, `MediaBox` và góc quay `/Rotate` bất kỳ, thuật toán chuyển đổi tọa độ trên Frontend được chuẩn hóa qua 3 bước như sau:

```javascript
/**
 * Chuyển đổi tổng quát BBox từ PyMuPDF sang Pixel Viewport của PDF.js.
 * Sử dụng ma trận tọa độ thực tế của trang và chuẩn hóa hai góc BBox.
 *
 * @param {Object} viewport - Viewport của PDF.js (page.getViewport({ scale, rotation }))
 * @param {Array<number>} bboxPyMuPdf - [x0, y0, x1, y1] theo hệ tọa độ hiển thị của PyMuPDF
 * @param {Array<number>} pageView - [cropX0, cropY0, cropX1, cropY1] từ page.view của PDF.js
 * @param {number} pageRotation - Góc xoay của trang (/Rotate: 0, 90, 180, 270)
 * @returns {Object} { left, top, width, height } tính bằng pixel cho thẻ DOM highlight
 */
function transformBBoxToViewport(viewport, bboxPyMuPdf, pageView, pageRotation = 0) {
    const [x0, y0, x1, y1] = bboxPyMuPdf;
    const [cropX0, cropY0, cropX1, cropY1] = pageView;

    // 1. Ánh xạ tọa độ từ PyMuPDF về PDF User Space chuẩn dựa trên góc quay /Rotate của trang
    let pdf_x0, pdf_y0, pdf_x1, pdf_y1;
    const rot = ((pageRotation % 360) + 360) % 360;

    switch (rot) {
        case 0:
            pdf_x0 = cropX0 + x0;
            pdf_y0 = cropY1 - y1;
            pdf_x1 = cropX0 + x1;
            pdf_y1 = cropY1 - y0;
            break;
        case 90:
            pdf_x0 = cropX0 + y0;
            pdf_y0 = cropY0 + x0;
            pdf_x1 = cropX0 + y1;
            pdf_y1 = cropY0 + x1;
            break;
        case 180:
            pdf_x0 = cropX1 - x1;
            pdf_y0 = cropY0 + y0;
            pdf_x1 = cropX1 - x0;
            pdf_y1 = cropY0 + y1;
            break;
        case 270:
            pdf_x0 = cropX1 - y1;
            pdf_y0 = cropY1 - x1;
            pdf_x1 = cropX1 - y0;
            pdf_y1 = cropY1 - x0;
            break;
        default:
            pdf_x0 = cropX0 + x0;
            pdf_y0 = cropY1 - y1;
            pdf_x1 = cropX0 + x1;
            pdf_y1 = cropY1 - y0;
    }

    // 2. Chuẩn hóa hai góc BBox: luôn đảm bảo [minX, minY, maxX, maxY] trước khi đưa vào PDF.js
    const normalizedPdfRect = [
        Math.min(pdf_x0, pdf_x1),
        Math.min(pdf_y0, pdf_y1),
        Math.max(pdf_x0, pdf_x1),
        Math.max(pdf_y0, pdf_y1)
    ];

    // 3. Sử dụng API PDF.js convertToViewportRectangle để chuyển đổi sang Pixel Viewport Canvas
    const rect = viewport.convertToViewportRectangle(normalizedPdfRect);
    const minX = Math.min(rect[0], rect[2]);
    const minY = Math.min(rect[1], rect[3]);
    const width = Math.abs(rect[2] - rect[0]);
    const height = Math.abs(rect[3] - rect[1]);

    return {
        left: `${minX}px`,
```

```
            top: `${minY}px`,
            width: `${width}px`,
            height: `${height}px`
        };
    }
```

5.2. Kết quả kiểm chứng thực tế và Giới hạn phạm vi tuyên bố (P0 & P1)

Theo tài liệu Lộ trình kỹ thuật (Mục 1.1 & 1.2), thuật toán toán học Affine 5 bước trong `app/pipeline/coordinate_helper.py` được báo cáo đạt **7/7 bài unit test thành công (0.049s)** với sai số lý thuyết **Max Error = 0.0000 pt** trên bộ 5 tệp kiểm chứng cố định (fixtures):

- `fixture_standard_a4.pdf`: Bản vẽ khổ A4, không xoay (Rot=0°), kiểm tra 4 góc trang và khung tên.
- `fixture_rotate_90.pdf`, `rotate_180.pdf`, `rotate_270.pdf`: Kiểm tra xử lý thuộc tính `/Rotate` trong PDF metadata.
- `fixture_cropbox_offset.pdf`: Kiểm tra bù trừ khi lề cắt kỹ thuật (CropBox) bị lệch so với MediaBox.

> ⚠ **GIỚI HẠN PHẠM VI TUYÊN BỐ & CẢNH BÁO KỸ THUẬT QUAN TRỌNG (P0 & P1):**
> 1. **Không suy rộng thành chính xác tuyệt đối trên mọi bản vẽ:** Kết quả 7/7 bài unit test và sai số 0.0000 pt chỉ có giá trị trên tập 5 tệp kiểm chứng mẫu (fixtures) được thiết kế có chủ đích. Bản vẽ CAD thực tế ngoài xưởng có thể phát sinh các cấu trúc ma trận biến đổi lồng nhau (nested transforms), đơn vị đo UserUnit phi tiêu chuẩn hoặc lề trang dị biệt.
> 2. **Chưa có mã nguồn và log kiểm thử độc lập:** Các chỉ số 7/7 tests hiện tại được ghi nhận dưới dạng số liệu do tài liệu nội bộ báo cáo; chưa có mã nguồn đầy đủ và log kiểm thử đi kèm trong tài liệu để các bên xác nhận độc lập.
> 3. **Unit test backend KHÔNG thay thế được Integration Test hiển thị Highlight:** Bộ unit test backend chỉ kiểm tra công thức tính toán đại số trên dòng lệnh, hoàn toàn chưa thể chứng minh khung highlight hiển thị trực quan khớp chính xác trên giao diện Web PDF.js.
> 4. **Yêu cầu bắt buộc xây dựng Integration Test Highlight (Giai đoạn 2):** Bắt buộc phải triển khai bộ kiểm thử tích hợp `tests/test_pdfjs_highlight_integration.py` (sử dụng Headless Browser / Playwright) để tải thực tế các bản vẽ, render trên PDF.js ở các tỷ lệ phóng thu (50%, 100%, 150%, 200%) và 4 góc xoay (0°, 90°, 180°, 270°), đo đạc tọa độ DOM thực tế của thẻ `div.highlight-box` so với nét chữ trên Canvas hiển thị.

PHẦN 6. TẦNG ĐỌC FILE (INGESTION): ROUTER 4 NHÁNH PDF BẢN VẼ, MULTI-OCR VÀ BỘ ĐỌC OFFICE MỞ RỘNG

Tầng Ingestion trong hệ thống được phân định rõ ràng giữa **Pipeline bóc tách bản vẽ kỹ thuật cốt lõi (Giai đoạn 1 & 2)** và **Bộ đọc mở rộng tài liệu văn phòng (Định hướng RAG tương lai)**:

6.1. Bộ định tuyến 4 nhánh tài liệu PDF (Đã hoàn thành 100% theo Mục 1.3)

Hệ thống không sử dụng các quy tắc cảm tính thô sơ (như đếm dưới 200 ký tự) mà vận hành **Bộ định tuyến 4 nhánh tài liệu PDF (`app/pipeline/document_router.py` - `[x] ĐÃ XONG` theo báo cáo nội bộ)**. Module này phân tích tổng hợp 3 chỉ số kỹ thuật của tệp: **mật độ văn bản (`text_density`), số lượng đối tượng hình học vector (`vector_primitives`: đường nét, khung vẽ CAD), và tỷ lệ diện tích ảnh quét (`raster_image`)**. Theo số liệu ghi nhận trong báo cáo nội bộ, việc đo kiểm trên **tập mẫu 15 file PDF thử nghiệm** đạt tỷ lệ phân loại **15/15 file đúng** (~300ms/file). *Lưu ý (P1):* Kết quả này mang tính chất ghi nhận trên tập mẫu thử nghiệm giới hạn, không nên suy rộng thành hệ thống chính xác tuyệt đối trên mọi bản vẽ thực tế; việc đối soát độc lập cần có mã nguồn và log kiểm thử đầy đủ đi kèm.

Nhánh tài liệu	Công cụ & Phương pháp đọc	Trạng thái	Chiến lược xử lý kỹ thuật
1. PDF Text thật *(Văn bản số có text stream)*	PyMuPDF (`fitz`) `text_extractor.py`	[x] ĐÃ XONG	Bóc trực tiếp text stream & bbox trong < 50ms/trang; không render ảnh, không tốn VRAM.
2. Bản vẽ CAD Vector *(Chứa vector primitives)*	PyMuPDF lấy text vector; Vùng nét vẽ đưa sang OCR	[x] ĐÃ XONG (Text) [ ] CHƯA LÀM (SHX)	Bóc tách Khung tên góc dưới-phải và bảng BOM phía trên khung tên; xử lý text CAD trong 15ms – 76ms.
3. PDF Scan Văn bản *(Ảnh scan chữ in tài liệu)*	Multi-OCR Adapter `ocr_adapter.py` (Mục 2.4)	[ ] CHƯA LÀM	Tiền xử lý OpenCV (deskew, CLAHE, Otsu) $\rightarrow$ **PaddleOCR CPU làm engine chính**; tự động kích hoạt **Tesseract (vie+eng) dự phòng** khi nghi ngờ nhầm số.
4. Bản vẽ Scan Mờ / Điểm mù *(Scan mờ, nét đè số đo)*	Multi-OCR + Cứu viện AI `ai_router.py` (Mục 3.3)	[ ] CHƯA LÀM	2-Pass: Cắt crop 300–400 DPI; nếu OCR confidence < 0.8 và có User Consent $\rightarrow$ gọi Gemini Rescue cứu viện ô mờ.
5. Bảng tính Excel (.xlsx, .xls) *(Định hướng mở rộng RAG)*	`pandas` + `openpyxl` (cho `.xlsx`) / `xlrd` (cho `.xls` cũ) `office_extractor.py`	[MỞ RỘNG]	Kế hoạch mở rộng RAG: Đọc từng sheet, bảo toàn hàng/cột, xuất bảng Markdown; dùng `openpyxl` cho tệp Excel mới (`.xlsx`) và thư viện `xlrd` cho tệp Excel nhị phân đời cũ (`.xls`); gắn chặt metadata `sheet_name` và `row_index` .
6. Văn bản Office (.docx, .pptx) *(Định hướng mở rộng RAG)*	`python-docx` , `python-pptx`	[MỞ RỘNG]	Trích xuất toàn bộ văn bản trong các đoạn văn, bảng biểu và ô ghi chú phục vụ đánh chỉ mục RAG.

6.2. Kế hoạch Module Multi-OCR Adapter (Giai đoạn 2 - Chưa làm)

Theo Lộ trình kỹ thuật Mục 2.4, module `app/pipeline/ocr_adapter.py` là hạng mục **CHƯA LÀM (`[ ] CHƯA LÀM`)**. Nhiệm vụ cốt lõi là kết hợp sức mạnh của 2 engine mã nguồn mở chạy 100% trên CPU:

- **Engine chính: PaddleOCR CPU:** Nhận diện cực nhanh các khối văn bản và bảng biểu kỹ thuật thông dụng.
- **Engine dự phòng: Tesseract 5 (vie+eng):** Tự động kích hoạt đối soát khi PaddleOCR trả về điểm tin cậy thấp hoặc nghi ngờ đọc sai các cặp ký tự số dễ nhầm lẫn trong cơ khí (ví dụ: `3` vs `8` , `0` vs `O` vs `Ø`).

6.3. Cơ chế Cache có phiên bản (Versioned Cache Key - Chưa làm theo Mục 2.5)

Hạng mục quản lý ảnh crop và sinh khóa cache `app/pipeline/evidence_store.py` là công việc **GIAI ĐOẠN 2 CHƯA LÀM (`[ ] CHƯA LÀM`)** theo Mục 2.5 của Lộ trình:

- **Mục tiêu:** Lưu trữ ảnh crop từng ô làm bằng chứng xác thực (evidence); sinh khóa băm định danh duy nhất để **không bao giờ phải OCR lại các file bản vẽ cũ không đổi**.
- **Cơ chế tự động vô hiệu hóa cache:** Khi cập nhật phiên bản pipeline hoặc thay đổi tham số cấu hình OCR/AI, chuỗi băm sẽ thay đổi, buộc hệ thống tự động quét lại mà không cần xóa cache thủ công.

```python
import hashlib
import json

def generate_versioned_cache_key(file_hash: str, page: int, pipeline_version: str, ocr_config: dict) -> str:
    """Kế hoạch Mục 2.5: Khóa cache sha256(file_hash + page + pipeline_version + config_hash)."""
    config_str = json.dumps(ocr_config, sort_keys=True)
    config_hash = hashlib.sha256(config_str.encode()).hexdigest()[:8]
    raw_key = f"{file_hash}:p{page}:{pipeline_version}:{config_hash}"
    return hashlib.sha256(raw_key.encode()).hexdigest()
```

PHẦN 7. TẦNG RETRIEVAL & RAG (ĐỊNH HƯỚNG MỞ RỘNG TƯƠNG LAI - CHƯA THUỘC PHẠM VI NGHIỆM THU V2.8)

*Lưu ý quan trọng: Toàn bộ kiến trúc Hybrid Search (BM25 + BGE-M3), Tái xếp hạng Reranker và cơ sở dữ liệu vector ChromaDB dưới đây được thiết kế như **định hướng mở rộng cho phân hệ tài liệu hỗn hợp trong tương lai**, KHÔNG coi là công nghệ đã có và CHƯA thuộc phạm vi nghiệm thu của Lộ trình v2.8 hiện hành.*

Hệ thống RAG nâng cao khi được kích hoạt sẽ vận hành theo mô hình **Hybrid Search 2 Tầng + Cross-Encoder Reranker** nhằm giải quyết bài toán tra cứu văn bản kỹ thuật và các mã mác thép cơ khí:

7.1. Chiến lược chia Chunk tối ưu cho Tài liệu Kỹ thuật (Định hướng RAG)

- **Đoạn văn bản thường:** Chunk 1000–1500 ký tự (~300–400 token), khoảng chồng lấn (overlap) 150–200 ký tự. Bắt buộc cắt theo ranh giới đoạn văn hoặc dấu chấm câu, cấm cắt ngang giữa câu.
- **Bảng biểu và Bảng tính Excel:** Giữ nguyên 1 bảng trong 1 chunk nếu bảng ngắn (< 30 dòng). Nếu bảng dài nhiều trang, chia theo từng cụm 15 dòng nhưng **bắt buộc lặp lại dòng tiêu đề (Header row) ở đầu mỗi chunk** để mô hình embedding hiểu được ngữ cảnh của từng ô dữ liệu.
- **Ảnh kỹ thuật sau OCR:** Mỗi ảnh / biểu đồ = 1 chunk độc lập kèm mô tả tóm tắt nội dung trực quan.

7.2. Quy trình Truy vấn 4 bước (Hybrid Retrieval Pipeline - Định hướng RAG)

1. **Bước 1 - Viết lại câu hỏi (Query Rewriting):** Mở rộng thêm 2 biến thể thuật ngữ đồng nghĩa.
2. **Bước 2 - Tìm kiếm lai (Hybrid Search):**
   - **Vector Search:** Dùng model `BAAI/bge-m3` (1024 chiều, đa ngữ) truy vấn trong ChromaDB lấy Top 20 chunk ngữ nghĩa.
   - **BM25 Keyword Search:** Dùng thuật toán BM25 (`rank-bm25`) lấy Top 20 chunk khớp chính xác mã vật liệu (`S45C` , `SUS304`), số hiệu bản vẽ, tên phụ tùng.
   - **Khử trùng lặp:** Hợp nhất 2 danh sách thành tập 25–30 chunks duy nhất.
3. **Bước 3 - Tái xếp hạng (Cross-Encoder Reranking):** Đưa qua mô hình `bge-reranker-v2-m3` (chạy local trên GPU) chấm lại điểm phù hợp, lọc ra **Top 5 chunk điểm cao nhất**.
4. **Bước 4 - Kiểm tra Ngưỡng từ chối (Confidence Threshold Gate - Kế hoạch RAG mở rộng):** Đưa ra ngưỡng thử nghiệm ban đầu (giả định 0.3). Khi kích hoạt phân hệ RAG trong tương lai, ngưỡng này **bắt buộc phải được hiệu chuẩn thực nghiệm trên bộ dữ liệu kiểm thử chuẩn (Golden QA dataset)** để đạt tỷ lệ cân bằng tối ưu giữa độ chính xác và khả năng từ chối an toàn; quy tắc này không áp dụng cho CAD Copilot hiện tại (vốn vận hành trực tiếp trên dữ liệu trung gian JSON đã bóc tách từ bản vẽ).

PHẦN 8. CƠ CHẾ SINH CÂU TRẢ LỜI: OLLAMA TEXT COPILOT TRÊN DỮ LIỆU TRUNG GIAN & PROMPT CHỐNG BỊA

8.1. Chatbot Copilot Bản vẽ trong phạm vi Lộ trình v2.8 (Giai đoạn 4 - Mục 4.1)

Theo đúng Lộ trình kỹ thuật, **Giai đoạn 4 không yêu cầu pipeline RAG phức tạp mà chỉ yêu cầu sử dụng Ollama Text (Qwen2.5) để chuẩn hóa JSON và vận hành Chatbot Copilot dựa trên dữ liệu trung gian chuẩn đã trích xuất từ bản vẽ**:

- **Nguồn dữ liệu của Chatbot:** Trợ lý Copilot đọc trực tiếp từ cấu trúc `result_json` trong PostgreSQL của bản vẽ đang mở (Title Block, danh sách linh kiện bảng BOM, các kích thước Ø/± và sai lệch).
- **Lợi ích vượt trội:** Phản hồi nhanh (mục tiêu 1–2 giây tùy kích thước dữ liệu bản vẽ), không tiêu tốn thêm VRAM cho mô hình Embedding/Reranker, hạn chế tối đa nguy cơ ảo giác vì chỉ đối chiếu trên dữ liệu máy vừa bóc tách.
- **Tương tác 2 chiều trên Web Review:** Khi kỹ sư hỏi: *"Bản vẽ này có bao nhiêu chi tiết làm bằng thép S45C?"*, Chatbot trả lời ngay số lượng và tự động kích hoạt highlight các dòng BOM tương ứng trên màn hình PDF.js.

8.2. Prompt Hệ thống chuẩn mực cho Chatbot Copilot Bản vẽ

```
Bạn là Trợ lý Kỹ thuật Copilot của xưởng cơ khí BOSOL AI.
Nhiệm vụ của bạn là trả lời câu hỏi của kỹ sư CHỈ DỰA TRÊN DỮ LIỆU TRUNG GIAN ĐÃ TRÍCH XUẤT TỪ BẢN VẼ DƯỚI ĐÂY.

NGUYÊN TẮC BẮT BUỘC:
1. Tuyệt đối không suy đoán số đo, kích thước hoặc vật liệu không có trong dữ liệu trích xuất.
2. Dẫn chiếu rõ ràng số thứ tự linh kiện (Item), mã chi tiết (Part Number) hoặc mã bản vẽ.
3. Nếu kỹ sư hỏi thông tin không có trong bảng BOM hoặc khung tên, trả lời ngắn gọn:
   "Dữ liệu bóc tách của bản vẽ này không chứa thông tin bạn yêu cầu."

=== DỮ LIỆU TRÍCH XUẤT TRUNG GIAN (JSON) ===
{extracted_drawing_json}
============================================

CÂU HỎI CỦA KỸ SƯ: {user_question}
CÂU TRẢ LỜI COPILOT:
```

8.3. Tham số sinh & Định hướng Prompt Grounding cho RAG Mở Rộng

- **Nhiệt độ (Temperature):** 0.1 (gần như triệt tiêu tính ngẫu nhiên, buộc mô hình bám sát dữ liệu).

- **Định dạng trích nguồn RAG mở rộng:** Khi phân hệ RAG mở rộng được kích hoạt, hệ thống sẽ sử dụng mẫu prompt ép trích nguồn `[Tên file, Trang số X]` hoặc `[Tên file, Sheet Y, Dòng Z]` kèm nút bấm nhảy trang trực tiếp.

PHẦN 9. QUY TRÌNH HỆ THỐNG TOÀN DIỆN TỪ ĐẦU ĐẾN CUỐI & NGUYÊN TẮC PHÊ DUYỆT TRƯỚC KHI XUẤT ERP

1. **Tiếp nhận tài liệu & Phân loại:** Kỹ sư tải file PDF bản vẽ lên Web UI. `document_router.py` phân tích trong < 50ms: phân loại chính xác thành PDF Text, CAD Vector, Scan văn bản hoặc Bản vẽ scan mờ (100% độ chính xác).
2. **Tạo Job & Kiểm tra Cache:** Tạo bản ghi trong PostgreSQL với `processing_status = 'queued'` và `review_status = 'pending'`. Kiểm tra cache có phiên bản: nếu trùng khớp khóa băm `sha256(file_hash + page + pipeline_ver + config_hash)`, trả kết quả tức thì (0ms).
3. **Trích xuất CAD & Cứu viện AI (Phân hệ cốt lõi):**
   - Đọc Khung tên, BOM từng ô và Kích thước bằng PyMuPDF hoặc Multi-OCR Adapter.
   - Nếu gặp điểm mù OCR (< 0.8), kiểm tra User Consent và hạn mức ngân sách $\rightarrow$ gọi Gemini Rescue Layer cứu ô mờ.
   - Nếu có bất đồng OCR vs Gemini: Lưu cả 2 ứng viên vào danh sách `candidates` bình đẳng, gắn cờ cảnh báo bất đồng trên UI.
   - Máy tính hoàn tất: Cập nhật `processing_status = 'completed'`, chuyển `review_status = 'in_review'`.
4. **Đánh chỉ mục RAG (Định hướng mở rộng tương lai):** Chỉ áp dụng khi mở rộng đọc kho tài liệu tổng hợp và bảng tính Excel.
5. **Kỹ sư kiểm duyệt trên Web Review Split-View 50/50:** Kỹ sư đối chiếu từng ô BOM và kích thước với ảnh chụp gốc. Bấm chọn ứng viên đúng qua Discrepancy Card.
6. **Khóa an toàn & Phê duyệt xuất ERP:** Khi toàn bộ các ô đã được đối chiếu (`pending_count == 0`), hệ thống **tuyệt đối không tự động chuyển sang** `approved`. Kỹ sư có thẩm quyền bắt buộc phải thực hiện **hành động phê duyệt chủ động và tường minh** (bấm nút *"Xác nhận & Phê duyệt bản vẽ"* có xác thực danh tính người duyệt `reviewed_by`). Backend kiểm tra tính hợp lệ của toàn bộ các trường bắt buộc; khi đạt đủ điều kiện, backend mới cập nhật `review_status = 'approved'` và mở khóa nút **Xuất dữ liệu ERP**.

PHẦN 10. CẤU TRÚC THƯ MỤC MÃ NGUỒN VÀ ĐẶC TẢ CÁC MODULE THEO PHÂN KỲ

```
drawing-ai/
├── app/
│   ├── main.py                    # Entry point FastAPI, CORS, static routes Web UI
│   ├── config.py                  # Cấu hình AI_ROUTER, GEMINI_MODEL_ID động, OLLAMA_TEXT, OCR
│   │
│   ├── api/                       # REST API Routers
│   │   ├── documents.py           # Upload PDF, xem tài liệu, tải kết quả
│   │   ├── jobs.py                # Truy vấn tiến độ job, lưu kết quả kỹ sư review, khóa xuất ERP
│   │   ├── chat.py                # [/] API Chatbot Copilot hỏi đáp trên dữ liệu trung gian bản vẽ (Mục 4.1)
│   │   └── worker_gateway.py      # API claim, heartbeat & submit cho worker qua LAN
│   │
│   ├── pipeline/                  # Pipeline xử lý đa phương thức (Local-First Hybrid)
│   │   ├── coordinate_helper.py   # [x] ĐÃ XONG: Toán học Affine (7/7 tests fixtures theo báo cáo)
│   │   ├── document_router.py     # [x] ĐÃ XONG: Router 4 nhánh (15/15 file mẫu theo báo cáo)
│   │   ├── text_extractor.py      # [x] ĐÃ XONG: Lõi bóc tách Text-First PyMuPDF (15ms - 76ms, 0 VRAM)
│   │   ├── ollama_adapter.py      # [/] ĐANG CẢI TIẾN: JSON Repair [x] & Refactor Ollama Text Copilot (Mục 4.1)
│   │   ├── ocr_adapter.py         # [ ] CHƯA LÀM: Multi-OCR (PaddleOCR CPU chính + Tesseract dự phòng - Mục 2.4)
│   │   ├── gemini_adapter.py       # [ ] CHƯA LÀM: Cứu viện Gemini API (SDK google-genai, Circuit Breaker - Mục 3.1 &
3.2)
│   │   ├── ai_router.py           # [ ] CHƯA LÀM: Bộ định tuyến AI (Consent, Budget, OCR conf < 0.8 - Mục 3.3)
│   │   ├── evidence_store.py      # [ ] CHƯA LÀM: Quản lý ảnh crop từng ô & Versioned Cache Key (Mục 2.5)
│   │   ├── image_enhancer.py      # Tiền xử lý scan mờ (OpenCV: CLAHE, Otsu, deskew)
│   │   ├── region_detector.py     # Định vị vùng Khung tên, BOM, Ghi chú
│   │   ├── drawing_parser.py      # [/] ĐANG CẢI TIẾN: Bóc tách Title Block, BOM từng ô và Kích thước (Ø, ±)
│   │   ├── office_extractor.py    # [MỞ RỘNG RAG]: Bóc tách Excel (sheet+dòng), Word (docx), PPT
│   │   ├── rag_chunker.py         # [MỞ RỘNG RAG]: Chia chunk thông minh giữ bảng biểu
│   │   ├── hybrid_retriever.py    # [MỞ RỘNG RAG]: Hybrid Search: Vector (bge-m3) + Từ khóa (BM25)
│   │   └── reranker_service.py    # [MỞ RỘNG RAG]: Cross-encoder reranker (bge-reranker-v2-m3)
│   │
│   ├── schemas/                   # Pydantic Schemas
│   │   ├── extraction.py          # [/] ĐANG CẢI TIẾN: Schema BBox BOM từng ô, candidates, dual status (Mục 2.2)
│   │   ├── chat_schema.py         # Schema hội thoại Chatbot & tin nhắn phản hồi
│   │   ├── job.py                 # Schema Job lifecycle & review status
│   │   └── rag.py                 # [MỞ RỘNG RAG]: Schema RAG chunk, query, citation metadata
│   │
│   ├── workers/
│   │   └── laptop_worker.py       # [/] ĐANG CẢI TIẾN: Worker Daemon hỗ trợ Direct DB Queue & HTTP Gateway (Mục 2.3)
│   │
│   ├── db/
│   │   ├── database.py            # [x] ĐÃ XONG: Kết nối PostgreSQL 16 cục bộ (localhost:5432/bosol_db)
│   │    ├── models.py                    # [/] ĐANG CẢI TIẾN: Dual-State model (processing/review), ai_requests,
user_consents (Mục 2.1)
│   │   └── chroma_client.py       # [MỞ RỘNG RAG]: Kết nối ChromaDB Local Persistent
│   │
│   └── static/                    # [x] ĐÃ XONG: Giao diện Web Review Split-View 50/50 & PDF.js
│       ├── index.html             # [x] Khung Split-View Trái Chatbot / Phải PDF.js Viewer
│       ├── app.js                 # [/] ĐANG CẢI TIẾN: Vẽ BBox Xanh/Vàng/Đỏ; Thẻ Discrepancy Card & Khóa ERP (Mục 4.2)
│       └── style.css              # [x] Định dạng giao diện Web Review
│
├── fixtures/
│   └── pdf_coords/                # [x] ĐÃ XONG: 5 tệp PDF fixtures chuẩn hóa (Rot=0°,90°,180°,270°,CropBox offset)
│
├── tests/                         # Bộ kiểm thử tự động hóa (Automated Tests & Benchmarks)
│   ├── test_coordinate_helper.py  # [x] ĐÃ XONG: Đo kiểm Affine: 7/7 bài test trên 5 fixtures theo báo cáo
│   ├── test_router_real_files.py  # [x] ĐÃ XONG: Đo kiểm Router 4 nhánh: 15/15 file mẫu đạt theo báo cáo (~300ms/file)
│   ├── test_json_repair.py        # [x] ĐÃ XONG: Đo kiểm phục hồi JSON AI: parse payload kiểm thử
│   ├── test_database_and_worker.py # [x] ĐÃ XONG: Đo kiểm DB Queue: SKIP LOCKED, Lease 300s, Reaper (3/3 test cases)
│   ├── test_gemini_rescue_crops.py # [ ] CHƯA LÀM: Đo kiểm cứu viện 10–20 ảnh crop mờ PDF_SCAN_BAN_VE (Mục 3.4)
│   └── run_accuracy_benchmark.py  # [ ] CHƯA LÀM: Benchmark 4 cấu hình bóc tách trên 40 file thực tế (Mục 4.3)
│
├── datasets/                      # Bộ dữ liệu kiểm chứng và đối chiếu
│   ├── ground_truth/              # 20–40 bản vẽ cơ khí kiểm chứng đối chiếu
│   └── golden_qa/                 # [MỞ RỘNG RAG]: 50–100 câu hỏi - đáp án vàng kiểm chứng RAG
│
├── requirements.txt               # pymupdf, paddleocr, pytesseract, google-genai, sqlalchemy, psycopg2...
└── README.md
```

PHẦN 11. CƠ CHẾ CÔ LẬP SỰ CỐ SUBPROCESS, KẾ HOẠCH CIRCUIT BREAKER VÀ TIMEOUT CHO GEMINI API

11.1. Kế hoạch Circuit Breaker và Retry cho Gemini API (Giai đoạn 3 - Chưa làm)

Theo Lộ trình kỹ thuật (Mục 3.1 & 3.2), cơ chế kết nối an toàn với Gemini API đang ở **GIAI ĐOẠN 3 CHƯA LÀM (`[ ] CHƯA LÀM`)**. Kế hoạch triển khai vào module `app/pipeline/gemini_adapter.py` được thiết kế chuẩn mực như sau:

1. **Circuit Breaker 3 trạng thái (CLOSED, OPEN, HALF-OPEN):** Nếu xảy ra 3 lần lỗi liên tiếp (tính sau khi đã retry cạn kiệt), Circuit Breaker chuyển sang trạng thái "OPEN" và tạm ngắt toàn bộ lời gọi lên Cloud trong 10 phút (600 giây). Sau thời gian ngắt, chuyển sang "HALF-OPEN" và cho phép duy nhất 1 request thăm dò để quyết định phục hồi hoặc tiếp tục ngắt.
2. **Retry có chọn lọc (Exponential Backoff):** Thử lại tối đa 2 lần với khoảng chờ tăng dần từ 2s đến 6s. Chỉ retry các lỗi mạng/timeout hoặc mã quá tải tạm thời (429/503); tuyệt đối không retry các lỗi 4xx client, sai API Key hay sai schema.
3. **Timeout ở tầng Client Transport 15 giây:** Cấu hình `types.HttpOptions(timeout=15.0)` khi khởi tạo Client (kiểm tra đơn vị theo phiên bản SDK); không đặt timeout bên trong `GenerateContentConfig`.
4. **Quy tắc mã nguồn thực tế:** Chuyển toàn bộ mã chạy thật sang file `.py` riêng kèm bộ kiểm thử tự động, không dựa vào mã mẫu trong tài liệu PDF làm mã xác thực chạy ngay.

```python
import time
import socket
import logging
from enum import Enum
from google import genai
from google.genai import types
from google.genai.errors import APIError
from tenacity import retry, stop_after_attempt, wait_exponential

logger = logging.getLogger(__name__)

class CircuitState(Enum):
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"

class GeminiCircuitBreaker:
    """Kế hoạch Mục 3.2: Circuit Breaker 3 trạng thái với cơ chế đếm lỗi sau retry và thăm dò phục hồi."""
    def __init__(self, failure_threshold: int = 3, recovery_time_sec: int = 600):
        self.failure_threshold = failure_threshold
        self.recovery_time_sec = recovery_time_sec
        self.consecutive_failures = 0
        self.circuit_open_until = 0.0
        self.state = CircuitState.CLOSED
        self._half_open_probe_in_flight = False

    def is_available(self) -> bool:
        now = time.time()
        if self.state == CircuitState.OPEN:
            if now >= self.circuit_open_until:
                self.state = CircuitState.HALF_OPEN
                self._half_open_probe_in_flight = False
                logger.info("CIRCUIT BREAKER -> HALF-OPEN: Cho phép 1 request thăm dò.")
            else:
                return False

        if self.state == CircuitState.HALF_OPEN:
            if self._half_open_probe_in_flight:
                return False
            self._half_open_probe_in_flight = True
            return True

        return True

    def record_success(self):
        self.consecutive_failures = 0
        self.circuit_open_until = 0.0
        self.state = CircuitState.CLOSED
        self._half_open_probe_in_flight = False

    def record_failure(self):
        """Ghi nhận thất bại sau khi đã retry cạn kiệt."""
        self._half_open_probe_in_flight = False
        if self.state == CircuitState.HALF_OPEN:
            self.state = CircuitState.OPEN
            self.circuit_open_until = time.time() + self.recovery_time_sec
            logger.warning(f"CIRCUIT BREAKER: Thăm dò thất bại, tiếp tục ngắt trong {self.recovery_time_sec}s!")
            return

        self.consecutive_failures += 1
        if self.consecutive_failures >= self.failure_threshold:
            self.state = CircuitState.OPEN
            self.circuit_open_until = time.time() + self.recovery_time_sec
            logger.warning(
                f"CIRCUIT BREAKER: Đạt ngưỡng {self.failure_threshold} lỗi liên tiếp sau retry, "
                f"tạm ngắt Gemini Cloud trong {self.recovery_time_sec}s!"
            )
```

```
circuit_breaker = GeminiCircuitBreaker(failure_threshold=3, recovery_time_sec=600)

def _is_retryable_error(exc: BaseException) -> bool:
    """Chỉ retry lỗi timeout/mạng hoặc lỗi máy chủ tạm thời 429/503."""
    if isinstance(exc, (TimeoutError, ConnectionError, socket.timeout)):
        return True
    if isinstance(exc, APIError):
        return exc.code in {429, 500, 502, 503, 504}
    return False

@retry(
    stop=stop_after_attempt(2),
    wait=wait_exponential(multiplier=1, min=2, max=6),
    retry=_is_retryable_error,
    reraise=True
)
def _call_gemini_with_transport_timeout(client: genai.Client, model_id: str, contents, config):
    """Gọi Gemini API với Client đã cấu hình transport timeout ở cấp genai.Client."""
    return client.models.generate_content(
        model=model_id,
        contents=contents,
        config=config
    )

def safe_call_gemini_with_retry(client: genai.Client, model_id: str, contents, config):
    """
    Thực thi an toàn có Circuit Breaker 3 trạng thái và Retry chọn lọc.
    Lưu ý: Client được khởi tạo với genai.Client(http_options=types.HttpOptions(timeout=15.0)).
    """
    if not circuit_breaker.is_available():
        raise RuntimeError("CIRCUIT_BREAKER_OPEN: Tạm dừng kết nối Gemini để tránh treo hàng đợi worker.")

    try:
        response = _call_gemini_with_transport_timeout(client, model_id, contents, config)
        circuit_breaker.record_success()
        return response
    except Exception as e:
        # Chỉ đếm thất bại sau khi đã thử lại hết số lần retry
        circuit_breaker.record_failure()
        logger.error(f"Lỗi kết nối Gemini API (sau retry / timeout / server): {e}")
        raise
```

11.2. Cách ly sự cố PDF bằng Subprocess độc lập trên Windows

```
import subprocess
import sys

def safe_render_and_ocr(pdf_path: str, timeout_sec: int = 60) -> str:
    """Cách ly sự cố crash thư viện C++ khi render PDF bằng Subprocess độc lập."""
    try:
        result = subprocess.run(
            [sys.executable, "-m", "app.pipeline.pdf_processor", pdf_path],
            capture_output=True,
            text=True,
            timeout=timeout_sec,
            check=True
        )
        return result.stdout
    except subprocess.TimeoutExpired:
        raise RuntimeError("ERR_TIMEOUT: Tiến trình xử lý PDF vượt quá thời gian cho phép")
    except subprocess.CalledProcessError as e:
        raise RuntimeError(f"ERR_PROCESS_FAILED: Tiến trình con gặp sự cố ({e.returncode}): {e.stderr}")
```

PHẦN 12. SCHEMA DỮ LIỆU: BOM CẤP TỪNG Ô, CANDIDATES ĐỐI CHIẾU VÀ ĐỊNH HƯỚNG RAG METADATA

12.1. Cấu trúc JSON Schema bóc tách Bản vẽ cơ khí (Đang cải tiến theo Mục 2.2)

Theo Lộ trình kỹ thuật Mục 2.2, tệp `app/schemas/extraction.py` đang được **cải tiến (** `[/] ĐANG CẢI TIẾN` **)** nhằm giải quyết triệt để lỗi logic: kỹ sư bấm chấp thuận nhưng giá trị xác minh bị thiếu (`accepted` mà `verified_value: null`). Trong cấu trúc mới, **mỗi ô trong bảng BOM (Item, Part Number, Quantity, Material) và Khung tên đều được lưu độc lập với 6 trường thông tin bắt buộc**:

1. `suggested_value` : Giá trị trích xuất do máy tính hoặc AI gợi ý.
2. `verified_value` : Giá trị đã được kỹ sư kiểm tra và chốt (khi đã duyệt, bắt buộc phải có giá trị chuẩn).
3. `bbox_pdf` : Tọa độ khung `[x0, y0, x1, y1]` trên trang PDF (chuẩn Points 72 DPI).
4. `source_method` : Phương pháp nhận diện (`text_stream_pymupdf` , `paddle_ocr_cpu` , `gemini_cloud_api`).
5. `evidence_crop_id` : Khóa ảnh crop chụp lại đúng vùng ô đó làm bằng chứng đối chiếu trực quan.
6. `approval_status` : Trạng thái duyệt độc lập của từng ô (`pending` , `approved` , `rejected`).

> **QUY TẮC AN TOÀN ERP:** Việc đạt `pending_count == 0` là **điều kiện cần nhưng chưa đủ**. Hệ thống tuyệt đối không tự động chuyển `review_status = 'approved'` . Quyết định phê duyệt bắt buộc phải do kỹ sư bấm nút xác nhận có chủ đích VÀ backend kiểm tra hợp lệ toàn bộ các trường bắt buộc (không rỗng, đúng định dạng). Chỉ khi đạt trạng thái `review_status == 'approved'` , API backend mới mở khóa quyền xuất dữ liệu sang hệ thống ERP (ngăn chặn hoàn toàn lỗi nghiệp vụ ở tầng máy tính).

```
{
  "$schema_version": "2.8.0",
  "document_id": "doc_2026_001",
  "job_id": "job_505",
  "processing_status": "completed",
  "review_status": "in_review",
  "metadata": {
    "file_name": "ban_ve_truc_truyen_scan_mo.pdf",
    "document_type": "scanned_drawing",
    "total_pages": 1,
    "pipeline_version": "2.8.0",
    "gemini_calls_count": 1,
    "gemini_cost_usd": 0.00015
  },
  "title_block": {
    "drawing_number": {
      "suggested_value": "SHAFT-2026-V1",
      "raw_text": "SHAFT-2026-V1",
      "page": 1,
      "bbox_pdf": [520.0, 40.5, 680.0, 65.0],
      "source_method": "text_stream_pymupdf",
      "evidence_crop_id": "crop_tb_01.png",
      "approval_status": "approved",
      "verified_value": "SHAFT-2026-V1",
      "reviewed_by": "engineer_01",
      "reviewed_at": "2026-10-24T15:20:00Z"
    },
    "part_name": {
      "suggested_value": "TRỤC TRUYỀN ĐỘNG",
      "raw_text": "TRUC TRUYEN DONG",
      "page": 1,
      "bbox_pdf": [520.0, 68.0, 750.0, 95.0],
      "source_method": "paddle_ocr_cpu",
      "evidence_crop_id": "crop_tb_02.png",
      "approval_status": "pending",
      "verified_value": null,
      "reviewed_by": null,
      "reviewed_at": null
    }
  },
  "bom": [
    {
      "item": {
        "suggested_value": 1,
        "page": 1,
        "bbox_pdf": [520.0, 110.0, 550.0, 130.0],
        "source_method": "paddle_ocr_cpu",
        "evidence_crop_id": "crop_bom_c01_item.png",
        "approval_status": "approved",
        "verified_value": 1
      },
      "part_number": {
        "suggested_value": "SHAFT-01",
        "raw_text": "SHAFT-01",
        "page": 1,
        "bbox_pdf": [550.0, 110.0, 660.0, 130.0],
        "source_method": "paddle_ocr_cpu",
        "evidence_crop_id": "crop_bom_c01_partno.png",
        "approval_status": "pending",
        "verified_value": null
      },
      "quantity": {
        "suggested_value": 1,
        "raw_text": "1",
        "page": 1,
        "bbox_pdf": [660.0, 110.0, 700.0, 130.0],
        "source_method": "paddle_ocr_cpu",
```

```
        "evidence_crop_id": "crop_bom_c01_qty.png",
        "approval_status": "approved",
        "verified_value": 1
      },
      "material": {
        "suggested_value": "S45C",
        "raw_text": "S45C",
        "page": 1,
        "bbox_pdf": [700.0, 110.0, 780.0, 130.0],
        "source_method": "paddle_ocr_cpu",
        "evidence_crop_id": "crop_bom_c01_mat.png",
        "approval_status": "pending",
        "verified_value": null
      }
    }
  ],
  "dimensions": [
    {
      "dimension_id": "dim_01",
      "type": "DIAMETER",
      "suggested_value": 35.0,
      "unit": "mm",
      "prefix": "Ø",
      "tolerance": {"upper": 0.0, "lower": -0.025},
      "page": 1,
      "bbox_pdf": [215.4, 340.2, 265.8, 355.0],
      "source_method": "gemini_cloud_api",
      "flag_ocr_vlm_disagreement": true,
      "candidates": [
        {"method": "paddle_ocr_cpu", "value": "Ø85", "note": "OCR gốc đọc nhầm nét cong"},
        {"method": "gemini_cloud_api", "value": "Ø35", "note": "Gemini cứu viện nhận diện nét mờ"}
      ],
      "evidence_crop_id": "crop_dim_01.png",
      "approval_status": "pending",
      "verified_value": null
    }
  ],
  "review_summary": {
    "total_fields": 7,
    "pending_count": 4,
    "approved_count": 3,
    "disagreement_count": 1,
     "computed_rule": "total_fields (7) = pending_count (4) + approved_count (3) [tính tự động từ toàn bộ các ô trong
title_block, bom và dimensions]"
  }
}
```


12.2. Cấu trúc JSON Schema Grounding Output (Định hướng mở rộng RAG)

(Áp dụng khi kích hoạt phân hệ RAG tra cứu tài liệu kỹ thuật hỗn hợp)

```
{
  "query": "Thép chế tạo trục chính là loại gì và độ cứng sau nhiệt luyện yêu cầu bao nhiêu?",
   "answer": "Thép chế tạo trục chính là S45C [ban_ve_truc.pdf, Trang 1]. Yêu cầu nhiệt luyện đạt độ cứng 45-50 HRC [thuyet_minh_ky_thuat.docx, Mục 3.2].",
  "confidence_score": 0.89,
  "sources": [
    {
      "file_name": "ban_ve_truc.pdf",
      "file_type": "cad_drawing",
      "page": 1,
      "sheet": null,
      "row": null,
      "content_snippet": "VẬT LIỆU: S45C; SỐ LƯỢNG: 01; TỶ LỆ 1:1"
    },
    {
      "file_name": "thuyet_minh_ky_thuat.docx",
      "file_type": "docx",
      "page": null,
      "sheet": null,
      "row": null,
       "content_snippet": "Mục 3.2. Yêu cầu tôi luyện: Bề mặt trục chính sau gia công phải tôi cao tần đạt độ cứng 45-50 HRC."
    }
  ]
}
```

PHẦN 13. BÀI TOÁN GIẢ ĐỊNH NGÂN SÁCH & PHÂN TÍCH CHI PHÍ GEMINI API CỨU VIỆN

*Lưu ý: Toàn bộ bảng tính và kịch bản dưới đây là **bài toán giả định tính toán lý thuyết** nhằm xác lập cơ chế quản lý ngân sách và trần chi phí; mức tiêu hao và chi phí USD thực tế sẽ được đo đạc, kiểm chứng và chuẩn hóa qua bài test cứu viện 10–20 crop mờ (Mục 3.4) và Benchmark 40 PDF thực tế (Mục 4.3).*

13.1. Các thông số giả định tính toán chi phí

- Mỗi tác vụ cứu viện ô mờ hoặc truy vấn RAG: Kèm prompt và ảnh crop $\approx$ 1.500 – 2.000 tokens Input.
- Đầu ra JSON có cấu trúc trung bình: $\approx$ 200 – 300 tokens Output.
- Tỷ giá quy đổi ước tính: **25.000 VNĐ / USD**.
- Toàn bộ tầng đọc file PyMuPDF, Multi-OCR, Database PostgreSQL và Ollama Text chạy 100% cục bộ trên Laptop: **Chi phí = 0 VNĐ**.

13.2. Bảng so sánh giả định chi phí gọi Gemini API theo lưu lượng hàng tháng

Số lượt cứu viện crop mờ mỗi tháng	Phương án A: Model Gemini Flash-Lite (Đơn giá giả định: $0.10/1M In, $0.40/1M Out)	Phương án B: Model Gemini Flash (Đơn giá giả định: $0.30/1M In, $2.50/1M Out)
100 lượt	~$0.03 (~800 VNĐ)	~$0.14 (~3.500 VNĐ)
300 lượt (Giai đoạn chạy thử)	**~$0.10 (~2.500 VNĐ)**	~$0.41 (~10.000 VNĐ)
1.000 lượt	~$0.32 (~8.000 VNĐ)	~$1.35 (~34.000 VNĐ)
5.000 lượt	~$1.60 (~40.000 VNĐ)	~$6.75 (~168.000 VNĐ)
10.000 lượt	**~$3.20 (~80.000 VNĐ)**	~$13.50 (~337.000 VNĐ)

KẾT LUẬN GIẢ ĐỊNH VỀ NGÂN SÁCH: Dưới giả định tính toán, với trần ngân sách kiểm soát **100.000 VNĐ / tháng** và chính sách chỉ gọi Cloud khi OCR địa phương gặp điểm mù (< 0.8), hệ thống hoàn toàn đảm bảo khả năng cứu viện hàng ngàn ô số đo mờ mỗi tháng mà không gây áp lực tài chính cho nhà máy. Số liệu thực tế sẽ được kiểm chứng qua module `ai_requests`.

PHẦN 14. KẾ HOẠCH KIỂM THỬ (BENCHMARK CAD 4 CẤU HÌNH TRÊN 40 PDF) & LỘ TRÌNH THỰC HIỆN CHI TIẾT

14.1. Kế hoạch kiểm thử trọng tâm: Benchmark 4 Cấu hình bóc tách Bản vẽ CAD (40 file thực tế)

Theo Lộ trình kỹ thuật Mục 4.3, để đánh giá chính xác hiệu quả giữa tốc độ, độ chính xác, mức chiếm VRAM và chi phí API thực tế của giải pháp Local-First Hybrid, hệ thống thiết lập bộ script đo kiểm tự động `tests/run_accuracy_benchmark.py` chạy trên tập **40 bản vẽ kỹ thuật cơ khí thực tế** (gồm CAD vector nguyên bản, scan rõ nét, scan mờ/nhiễu, và bản vẽ có góc xoay/lệch CropBox):

Cấu hình đo kiểm	Các thành phần Engine tham gia	Mục tiêu đo lường thực nghiệm	Chỉ số kỳ vọng đo kiểm
Cấu hình A (OCR thuần)	PaddleOCR CPU (chính) + Tesseract 5 Fallback (khi độ tin cậy thấp hoặc nghi ngờ nhầm số).	Đo năng lực bóc tách cơ sở khi hoàn toàn không có sự hỗ trợ của LLM/VLM.	• Độ chính xác: ~75% - 82% • Latency: < 800ms/trang • Chi phí: 0 VNĐ • VRAM: 0 MB
Cấu hình B (A + Ollama)	Cấu hình A + Ollama Text (Qwen2.5-7B thử nghiệm, chạy local trên GPU trạm) để chuẩn hóa cấu trúc JSON.	Đo năng lực tự phục hồi cú pháp và chuẩn hóa dữ liệu offline 100%; kiểm tra mức chiếm VRAM.	• Độ chính xác: ~85% - 90% • Latency: ~2s - 4s/trang • Chi phí: 0 VNĐ • VRAM: ~4.0 - 4.2 GB
Cấu hình C (A + Gemini)	Cấu hình A + Google Gemini API (`GEMINI_MODEL_ID` động) cứu viện các vùng crop OCR điểm mù (< 0.8) qua SDK `google-genai`.	Đo năng lực vượt qua điểm mù scan mờ và ghi nhận chi phí USD thực tế qua bảng `ai_requests`.	• Độ chính xác: ~94% - 97% • Latency: ~1.5s - 2.5s/crop • Chi phí thực tế: Xác minh qua test • VRAM: 0 MB
Cấu hình D (Full Hybrid)	Phối hợp đầy đủ: Text Stream PyMuPDF + Multi-OCR + Ollama Text + Gemini Rescue Layer khi có điểm mù.	Đo năng lực toàn diện của hệ thống trong môi trường sản xuất công nghiệp thực tế.	• Độ chính xác: **> 98%** • Latency P50: < 2.5s • Chi phí: Kiểm soát theo ngân sách • VRAM: An toàn trong GPU 6GB

> **NGUYÊN TẮC ĐO KIỂM THỰC NGHIỆM:** Mọi dự báo chi phí trước đó chỉ là bài toán giả định. Script `tests/run_accuracy_benchmark.py` sẽ xuất báo cáo thống kê chính thức đo lường 4 thông số: **(1) Độ chính xác Accuracy, (2) Độ trễ Latency P50/P95, (3) Chi phí API thực tế phát sinh (USD), (4) Mức chiếm VRAM máy trạm**.

14.2. Khung đánh giá chất lượng Phân hệ B (Định hướng RAG Mở Rộng - Chưa nghiệm thu v2.8)

(Áp dụng khi kích hoạt phân hệ RAG mở rộng tra cứu tài liệu hỗn hợp)

Chỉ số đo lường RAG	Phương pháp xác định	Mục tiêu kỳ vọng (Định hướng)
1. Retrieval Hit Rate	Tỷ lệ phần trăm câu hỏi mà đoạn văn bản chứa câu trả lời đúng nằm trong Top 5 chunk lấy về.	**> 90%** (Đảm bảo tìm kiếm không bỏ sót thông tin kỹ thuật).
2. Faithfulness	Tỷ lệ các ý trong câu trả lời được chứng minh trực tiếp từ ngữ cảnh trích dẫn (chống bịa).	**Mục tiêu $\ge$ 98%** trên tập Golden QA kiểm thử (hạn chế tối đa thông tin ngoài ngữ cảnh được cung cấp).
3. LLM-as-judge Score	Dùng một mô hình LLM độc lập chấm điểm câu trả lời so với đáp án vàng trên thang điểm 1–5.	**> 4.5 / 5.0**.

14.3. Vòng lặp sửa lỗi 3 bước cho RAG (Debug Triad - Định hướng mở rộng)

Khi có một câu hỏi bị trả lời sai hoặc không tìm thấy trong tài liệu tổng hợp, quy trình kiểm tra gồm 3 bước:

1. **Kiểm tra Chunk:** Bảng biểu có bị cắt ngang không? Dòng tiêu đề có bị mất không? $\rightarrow$ Điều chỉnh lại kích thước chunk và lặp lại Header row.
2. **Kiểm tra Retrieval:** Chunk đúng có nằm trong Top 20 của BM25 và Vector không? Reranker có chấm sai vị trí không? $\rightarrow$ Tinh chỉnh trọng số BM25 hoặc câu lệnh query rewrite.
3. **Kiểm tra Prompt:** LLM có suy diễn ngoài ngữ cảnh hay diễn đạt tối nghĩa không? $\rightarrow$ Siết chặt lại System Prompt và giảm nhiệt độ temperature về 0.1.

14.4. Những bài học kinh nghiệm thực chiến từ tập dữ liệu 561 PDF

- **Bài học 1 - Tài liệu ngoài lề lọt vào kho dữ liệu:** Khi nhập hàng loạt (bulk import), không được chỉ dựa vào tiêu đề file để phân loại. Bắt buộc phải trích xuất mẫu 2–3 trang đầu để kiểm tra nội dung; nếu tài liệu là tạp chí quảng cáo, hóa đơn linh tinh thì phải lọc bỏ ngay trước khi chunking.
- **Bài học 2 - Chunk cắt ngang bảng biểu:** Khi bảng tính Excel hoặc bảng kỹ thuật bị cắt đôi, các dòng ở chunk sau mất hoàn toàn ý nghĩa cột. Bắt buộc phải áp dụng quy tắc lặp Header row cho mọi chunk con của bảng.
- **Bài học 3 - Điểm mù của Vector Search thuần:** Vector search rất kém khi tìm các chuỗi số rời rạc hoặc mã mác thép (ví dụ: `Ø35k6`, `SUS304-2B`). Việc kết hợp thuật toán BM25 là bắt buộc, không được bỏ qua.

14.5. Lộ trình phát triển toàn diện & Bảng theo dõi tiến độ chi tiết (Đồng bộ theo Lộ trình v2.8/v2.9)

*Theo tài liệu quy chiếu Lo_trinh_phat_trien_va_tien_do_du_an.pdf, dự án đã **hoàn thành ~65% - 70%** khối lượng công việc nền tảng cốt lõi (Giai đoạn 1 đã xong 100%, Giai đoạn 2 đang triển khai, Giai đoạn 3 và 4 tiếp tục theo kế hoạch).*

```
gantt
    title Lộ Trình Phát Triển Toàn Diện Hệ Thống BOSOL AI
    dateFormat  YYYY-MM-DD
    section GĐ 1: Nền Tảng Lõi (ĐÃ XONG)
    Toán học Affine 5 bước & Fixtures Coords     :done, g1_1, 2026-10-15, 3d
    Router 4 nhánh & Trích xuất PyMuPDF Text     :done, g1_2, after g1_1, 4d
    Cơ sở dữ liệu PostgreSQL 16 & Job Queue      :done, g1_3, after g1_2, 3d
    Giao diện Web Review Split-View 50/50 & PDF.js:done, g1_4, after g1_3, 4d
    section GĐ 2: Chuẩn Hóa Schema & Multi-OCR (HIỆN TẠI)
    Tách Dual-State (2.1) & Schema BOM từng ô (2.2):active, g2_1, 2026-10-24, 2d
    Direct DB Queue Worker cho máy đơn (2.3)     :active, g2_3_worker, after g2_1, 2d
    Tích hợp Multi-OCR Paddle + Tesseract (2.4)  :      g2_4_ocr, after g2_3_worker, 3d
    Cache có phiên bản Versioned Cache Key (2.5) :      g2_5_cache, after g2_4_ocr, 2d
    section GĐ 3: AI Router & Gemini Rescue (TIẾP THEO)
    Adapter SDK google-genai & Circuit Breaker   :      g3_1, after g2_5_cache, 3d
    Bộ định tuyến AI Router (Consent & Budget)   :      g3_2, after g3_1, 3d
    Kiểm thử cứu viện 10-20 ảnh crop scan mờ     :      g3_3, after g3_2, 2d
    section GĐ 4: Chatbot Copilot & Nghiệm Thu Xưởng
    Ollama Text chuẩn hóa JSON & Chatbot Copilot :      g4_1, after g3_3, 3d
    Web Review: Thẻ Discrepancy Card & Khóa ERP  :      g4_2, after g4_1, 3d
    Benchmark 4 Cấu hình (40 files) & Bàn giao   :      g4_3, after g4_2, 4d
```

BẢNG THEO DÕI CHI TIẾT TỪNG HẠNG MỤC CÔNG VIỆC THEO 4 GIAI ĐOẠN

GIAI ĐOẠN 1: NỀN TẢNG TOÁN HỌC, XỬ LÝ TEXT-FIRST & HẠ TẦNG CỤC BỘ (ĐÃ HOÀN THÀNH 100%)

Mục tiêu: Đảm bảo độ chính xác tuyệt đối về hệ tọa độ và tốc độ bóc tách tức thì cho bản vẽ có sẵn text layer.

Mã	Hạng mục công việc	Trạng thái	Chú thích kết quả đầu ra thực tế (Output Result)
1.1	**Toán học hệ tọa độ ma trận Affine 5 bước** `app/pipeline/coordinate_helper.py`	[x] ĐÃ XONG	☑ **Kết quả:** Đã lập trình xong 294 dòng code tính toán Affine xuôi/ngược. ☑ **Đo kiểm theo báo cáo nội bộ:** Chạy `tests/test_coordinate_helper.py` đạt **7/7 bài test thành công** (0.049s) trên bộ 5 fixtures mẫu. (Cần lưu ý chưa kèm mã nguồn và log kiểm thử độc lập; chưa suy rộng ra mọi loại CAD). ☑ **Sai số chuyển đổi lý thuyết:** Max Error = `0.0000 pt` trên 5 file fixtures chuẩn.
1.2	**Bộ tạo tệp PDF Fixtures kiểm chứng tọa độ** `fixtures/pdf_coords/`	[x] ĐÃ XONG	☑ **Kết quả:** Đã tạo thành công 5 tệp PDF chuẩn hóa: • `fixture_standard_a4.pdf` (Chuẩn) • `fixture_rotate_90.pdf`, `rotate_180.pdf`, `rotate_270.pdf` • `fixture_cropbox_offset.pdf` (Lệch MediaBox).
1.3	**Bộ định tuyến 4 nhánh tài liệu PDF** `app/pipeline/document_router.py`	[x] ĐÃ XONG	☑ **Kết quả:** Phân tích text density, vector primitives, raster image. ☑ **Đo kiểm theo báo cáo nội bộ:** Chạy `tests/test_router_real_files.py` trên **tập 15 file PDF mẫu thử nghiệm** đạt kết quả phân loại **15/15 file**. Tốc độ trung bình: ~300ms/file (chưa suy rộng thành tuyệt đối trên mọi bản vẽ ngoài thực tế).
1.4	**Lõi bóc tách Text-First bằng PyMuPDF** `app/pipeline/text_extractor.py`	[x] ĐÃ XONG	☑ **Kết quả:** Trích xuất trực tiếp Text stream, Title block, BOM table và kích thước. ☑ **Đo kiểm thực tế:** Bóc tách tệp CAD và PDF Text hoàn tất trong **15ms – 76ms**, không tốn VRAM card đồ họa.
1.5	**Thuật toán sửa lỗi cú pháp JSON từ AI (JSON Repair)** `app/pipeline/ollama_adapter.py`	[x] ĐÃ XONG	☑ **Kết quả:** Hàm `repair_and_parse_json` phục hồi JSON bị thiếu ngoặc, dính text rác. ☑ **Đo kiểm thực tế:** Chạy `tests/test_json_repair.py` parse thành công 100% payload đơn hàng phức tạp.
1.6	**Cơ sở dữ liệu PostgreSQL 16 & Hàng đợi Job Queue** `app/db/database.py`, `app/db/models.py`	[x] ĐÃ XONG	☑ **Kết quả:** Đã kết nối PostgreSQL 16 cục bộ tại `localhost:5432/bosol_db`. ☑ **Đo kiểm theo báo cáo nội bộ:** Chạy `tests/test_database_and_worker.py` đạt **3/3 test cases kịch bản mẫu**: Nhận việc nguyên tử `SKIP LOCKED`, gia hạn Lease 300s, chặn ghi đè (409 Conflict) và Reaper dọn Zombie job thành công.
1.7	**Giao diện Web Review Split-View 50/50 & PDF.js** `app/static/index.html`, `app.js`, `style.css`	[x] ĐÃ XONG	☑ **Kết quả:** Dựng xong giao diện Split-View 50% Trái (Chatbot Copilot) và 50% Phải (PDF.js Viewer). ☑ **Tính năng:** Tự động vẽ khung Bounding Box đổi màu theo trạng thái (Xanh: OK, Vàng: Chờ duyệt, Đỏ: Bất đồng), liên kết 2 chiều bấm vào chat cuộn đến bản vẽ.

GIAI ĐOẠN 2: CHUẨN HÓA SCHEMA v2.8/v2.9, MULTI-OCR & DIRECT DB QUEUE (CÔNG VIỆC HIỆN TẠI)

Mục tiêu: Đồng bộ mã nguồn hiện tại theo đúng các cải tiến P0 & P1 của tài liệu đặc tả.

Mã	Hạng mục công việc	Trạng thái	Mục tiêu kỹ thuật & Kết quả đầu ra mong đợi
2.1	**Tách Mô hình Trạng thái Kép (Dual-State Model)** `app/db/models.py`	[/] ĐANG CẢI TIẾN	🔄 **Cần làm:** Tách cột `status` thành 2 cột độc lập: • `processing_status`: `queued`, `processing`, `completed`, `failed` • `review_status`: `pending`, `in_review`, `approved`, `rejected` • Thêm bảng `ai_requests` (lưu token/chi phí) và `user_consents`.
2.2	**Cập nhật Schema BBox từng ô bảng BOM** `app/schemas/extraction.py`	[/] ĐANG CẢI TIẾN	🔄 **Cần làm:** Khắc phục lỗi `accepted` mà `verified_value: null`. • Mỗi ô trong BOM (Item, Part Number, Qty, Material) lưu riêng: `suggested_value`, `verified_value`, `bbox_pdf`, `source_method`, `evidence_crop_id`, `approval_status`.
2.3	**Hỗ trợ Direct DB Queue Worker cho máy đơn** `app/workers/laptop_worker.py`	[/] ĐANG CẢI TIẾN	🔄 **Cần làm:** Cho phép Worker lấy việc trực tiếp từ PostgreSQL nội bộ qua kết nối DB (bỏ qua HTTP Gateway khi chạy All-in-One trên 1 laptop), giảm overhead và không lo lỗi kết nối mạng nội bộ.
2.4	**Tạo Module Multi-OCR Adapter (Paddle + Tesseract)** `app/pipeline/ocr_adapter.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Viết module gom 2 engine OCR: PaddleOCR CPU làm engine chính; tự động kích hoạt Tesseract OCR (vie+eng) khi độ tin cậy thấp hoặc nghi ngờ nhầm số 3/8, 0/O/Ø.
2.5	**Cơ chế Cache có phiên bản (Versioned Cache Key)** `app/pipeline/evidence_store.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Lưu ảnh crop từng ô; sinh khóa cache: `sha256(file_hash + page + pipeline_ver + config_hash)` để không OCR lại file cũ nhưng tự động quét lại khi đổi model/thuật toán.

GIAI ĐOẠN 3: TẦNG CỨU VIỆN GEMINI API & BỘ ĐỊNH TUYẾN AI ROUTER (KẾ HOẠCH TIẾP THEO)

Mục tiêu: Hóa giải các điểm mù OCR scan mờ mà không làm tràn VRAM laptop, kiểm soát chặt chẽ ngân sách.

Mã	Hạng mục công việc	Trạng thái	Mục tiêu kỹ thuật & Kết quả đầu ra mong đợi
3.1	**Adapter kết nối Gemini API bằng SDK `google-genai`** `app/pipeline/gemini_adapter.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Tích hợp SDK mới nhất `google-genai`; cấu hình `GEMINI_MODEL_ID` động (mặc định `gemini-2.5-flash` hoặc model Flash đang active); xuất JSON có cấu trúc (Structured Outputs); cấu hình timeout 15s thực tế ở cấp khởi tạo client qua `genai.Client(http_options=types.HttpOptions(timeout=15.0))` (đối soát đơn vị giây vs ms theo phiên bản SDK cài đặt; không đặt trong GenerateContentConfig); chuyển mã chạy thật sang file `.py` độc lập kèm automated tests.
3.2	**Hệ thống Circuit Breaker & Retry Exponential Backoff** `app/pipeline/gemini_adapter.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Circuit Breaker 3 trạng thái (CLOSED, OPEN, HALF-OPEN); đếm lỗi sau khi đã hết lượt retry; tự động ngắt kết nối Gemini trong 10 phút nếu lỗi liên tiếp 3 lần; phục hồi qua 1 request thăm dò; retry chọn lọc 2 lần cho lỗi mạng/timeout và 429/503 (loại trừ lỗi 4xx client).
3.3	**Bộ não định tuyến thông minh `ai_router.py`** `app/pipeline/ai_router.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Kiểm tra đồng thời 3 điều kiện: (1) OCR confidence < 0.8, (2) User Consent = True, (3) Chi phí ngày < 2 USD. Nếu thỏa mãn mới gửi crop sang Gemini. Lưu kết quả đối chiếu vào `candidates` bình đẳng.
3.4	**Thử nghiệm thực tế Gemini trên 10–20 crop khó** `tests/test_gemini_rescue_crops.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Cắt 20 ảnh crop số đo/bảng mờ trong `PDF_SCAN_BAN_VE` mà PaddleOCR đọc sai; đo tỷ lệ đọc đúng của Gemini, ghi nhận chi phí thực tế (USD/lần gọi).

GIAI ĐOẠN 4: OLLAMA TEXT COPILOT, WEB REVIEW NÂNG CAO & NGHIỆM THU (HOÀN THIỆN)

Mục tiêu: Đưa hệ thống vào ứng dụng thực tế tại nhà xưởng, nghiệm thu quy trình xuất ERP.

Mã	Hạng mục công việc	Trạng thái	Mục tiêu kỹ thuật & Kết quả đầu ra mong đợi
4.1	**Refactor Ollama: Tập trung Ollama Text cho Chatbot Copilot** `app/pipeline/ollama_adapter.py`, `app/api/chat.py`	[/] ĐANG CẢI TIẾN	🔄 **Cần làm:** Tắt chế độ ép chạy Ollama Vision gây tốn VRAM; sử dụng Ollama Text (Qwen2.5) để chuẩn hóa JSON và trả lời câu hỏi Chatbot dựa trên dữ liệu trung gian chuẩn đã trích xuất.
4.2	**Web Review: Thẻ Discrepancy Card & Khóa xuất ERP** `app/static/app.js`, `index.html`	[ ] CHƯA LÀM	📋 **Cần làm:** Thêm công tắc Consent bật/tắt gửi Gemini; hiển thị thẻ Discrepancy Card với 2 nút chọn nhanh `[A] OCR` hoặc `[B] Gemini`; **khóa nút xuất ERP** cho đến khi `review_status == 'approved'`.
4.3	**Chạy Benchmark 4 cấu hình trên 40 file thực tế** `tests/run_accuracy_benchmark.py`	[ ] CHƯA LÀM	📋 **Cần làm:** Đo kiểm so sánh 4 cấu hình: (A) OCR thuần, (B) A+Ollama, (C) A+Gemini, (D) A+Ollama+Gemini. Xuất bảng thống kê tỷ lệ chính xác, P50/P95 latency, chi phí USD và VRAM.
4.4	**Đóng gói nghiệm thu & Bàn giao thí điểm xưởng (Pilot)** `README.md`, Hướng dẫn SOP	[ ] CHƯA LÀM	📋 **Cần làm:** Viết tài liệu quy trình vận hành chuẩn cho kỹ sư xưởng; hướng dẫn chạy local qua mạng LAN `http://192.168.1.x:8000`; nghiệm thu bàn giao dự án.

14.6. Tổng kết danh mục file nguồn cần thao tác tiếp theo

Dựa trên tiến độ thực tế, bảng phân công nhiệm vụ cho các tệp mã nguồn trong dự án được xác định như sau:

Hành động	Các File Cần Tác Động	Mục Đích Cụ Thể
1. SỬA CHỮA (Refactor)	• `app/db/models.py` • `app/schemas/extraction.py` • `app/pipeline/ollama_adapter.py` • `app/workers/laptop_worker.py` • `app/static/app.js`	• Tách `processing_status` và `review_status`. • Bổ sung BBox & Evidence cho từng ô BOM. • Chuyển Ollama sang chế độ thuần Text. • Hỗ trợ Direct DB Queue Worker. • Thêm thẻ Discrepancy Card trên Web UI.
2. VIẾT MỚI (Create New)	• `app/pipeline/ocr_adapter.py` • `app/pipeline/gemini_adapter.py` • `app/pipeline/ai_router.py` • `app/pipeline/evidence_store.py` • `tests/test_gemini_rescue_crops.py`	• Kết hợp PaddleOCR + Tesseract Fallback. • Gọi Gemini bằng SDK `google-genai` kèm Circuit Breaker. • Định tuyến thông minh có kiểm soát ngân sách. • Quản lý ảnh crop và Versioned Cache Key. • Kiểm thử cứu viện điểm mù scan mờ thực tế.
3. GIỮ NGUYÊN THEO BÁO CÁO NỘI BỘ (Keep As-Is)	• `app/pipeline/coordinate_helper.py` • `app/pipeline/document_router.py` • `app/pipeline/text_extractor.py` • `fixtures/pdf_coords/*` • `tests/test_coordinate_helper.py`	• Đã vượt qua các bài unit test trên bộ tệp mẫu thử nghiệm cố định theo báo cáo nội bộ. • Tính toán tọa độ trên 5 fixtures và phân loại trên tập 15 file mẫu đạt kết quả tốt; cần tiếp tục kiểm chứng độc lập kèm log thực thi trên tập bản vẽ thực tế mở rộng.

PHẦN 15. KẾT LUẬN VÀ CÁC QUYẾT ĐỊNH THIẾT KẾ BẤT BIẾN (LOCAL-FIRST HYBRID ARCHITECTURE)

Bản đặc tả kỹ thuật xác lập lộ trình triển khai rõ ràng, tập trung dứt điểm vào **CAD Engine và Chatbot Copilot bản vẽ** theo **4 giai đoạn tuần tự chuẩn mực** của Lộ trình phát triển v2.8:

1. **Giai đoạn 1 - Nền tảng lõi (ĐÃ HOÀN THÀNH THEO BÁO CÁO NỘI BỘ):** Toán học Affine (7/7 tests fixture mẫu), Fixtures kiểm chứng, Bộ định tuyến 4 nhánh (15 file mẫu thử nghiệm), Trích xuất Text-first PyMuPDF, PostgreSQL DB Queue (3/3 tests mẫu), Web Review Split-View 50/50 & PDF.js (yêu cầu bổ sung integration test highlight).
2. **Giai đoạn 2 - Chuẩn hóa Schema & Multi-OCR (HIỆN TẠI - ĐANG THỰC HIỆN):** Tách mô hình Dual-State (`processing_status` & `review_status`, bảng `user_consents`, `ai_requests`); Cập nhật Schema BBox từng ô bảng BOM (Item, Part Number, Qty, Material kèm `suggested_value`, `verified_value`, `bbox_pdf`, `evidence_crop_id`); Hỗ trợ Direct DB Queue Worker; Module Multi-OCR Adapter (PaddleOCR CPU chính + Tesseract dự phòng); Cơ chế Cache có phiên bản (`evidence_store.py`).
3. **Giai đoạn 3 - AI Router & Tầng cứu viện Gemini (KẾ HOẠCH TIẾP THEO):** Tích hợp SDK `google-genai` với `GEMINI_MODEL_ID` động; Thiết lập Circuit Breaker tự ngắt và Retry lũy thừa; Bộ não định tuyến `ai_router.py` kiểm soát 3 điều kiện (OCR conf < 0.8, User Consent = True, Chi phí ngày < 2 USD); Kiểm thử thực tế trên 10–20 ảnh crop mờ.
4. **Giai đoạn 4 - Ollama Text Copilot, Web Review Nâng cao & Nghiệm thu xưởng (HOÀN THIỆN):** Refactor Ollama Text (Qwen2.5) chuẩn hóa JSON và trợ lý Copilot hỏi đáp trên dữ liệu trung gian trích xuất từ bản vẽ; Web Review hoàn thiện thẻ Discrepancy Card và khóa nút xuất ERP khi chưa duyệt; Benchmark 4 cấu hình trên 40 file thực tế đo accuracy, P50/P95, chi phí, VRAM; Đóng gói SOP và bàn giao thí điểm xưởng.

> **ĐỊNH VỊ PHÂN HỆ RAG NÂNG CAO:** Toàn bộ phân hệ RAG tài liệu hỗn hợp (Hybrid Search BM25 + BGE-M3, Reranker, ChromaDB, đọc file Office Excel/Word) là **bước mở rộng tiếp theo sau khi hệ thống CAD Engine đã được nghiệm thu bàn giao thành công** tại nhà máy sản xuất.

Ba quyết định thiết kế bất biến của dự án

1. **Sức mạnh Local-First tối đa trên 01 Laptop nội bộ:** 100% quá trình đọc file Text, CAD Vector, OCR cơ sở và Ollama Text chạy hoàn toàn cục bộ trong xưởng (Chi phí thuê máy chủ = 0 VNĐ, bảo mật tuyệt đối dữ liệu thiết kế).
2. **Cloud Rescue Layer có kiểm soát chặt chẽ:** Gemini API chỉ xuất hiện khi OCR cục bộ gặp điểm mù (< 0.8), có sự đồng ý của kỹ sư (User Consent) và luôn nằm dưới trần ngân sách kiểm soát. Mức chi phí thực tế sẽ được xác minh qua đo kiểm.
3. **Nguyên tắc Kỹ sư là Trọng tài Tối cao (Zero Blind Trust):** Mọi số đo gia công trước khi xuất sang ERP bắt buộc phải được kỹ sư bấm duyệt (`review_status == 'approved'`). Hệ thống khóa cứng thao tác xuất dữ liệu khi còn bất kỳ ô nào chưa được xác thực.